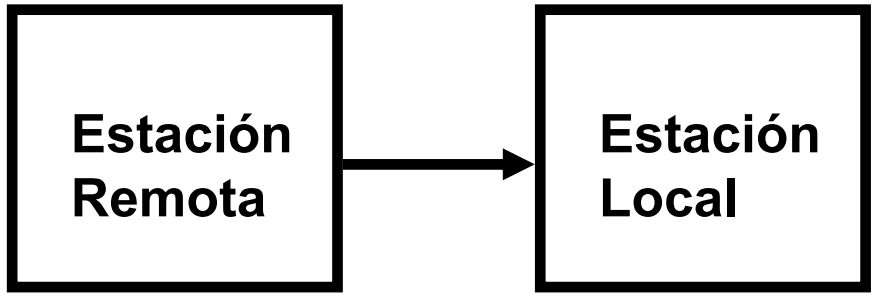
Puerto Serie

Modos de Transmisión

- ***Simplex***
- ***Half Duplex***
- ***Full Duplex***

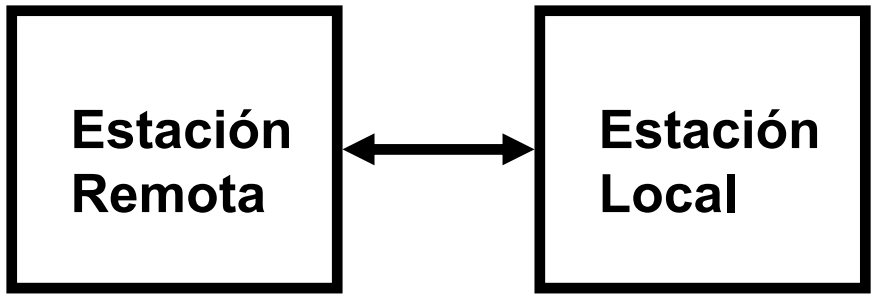
Transmisión Simplex

- *Transmisión en un solo sentido*



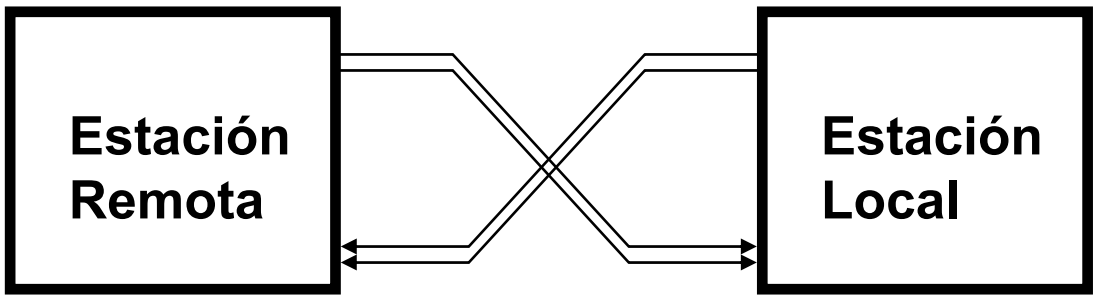
Transmisión Half Duplex

- *Transmisión en un ambos sentidos pero no en forma simultánea.*



Transmisión Full Duplex

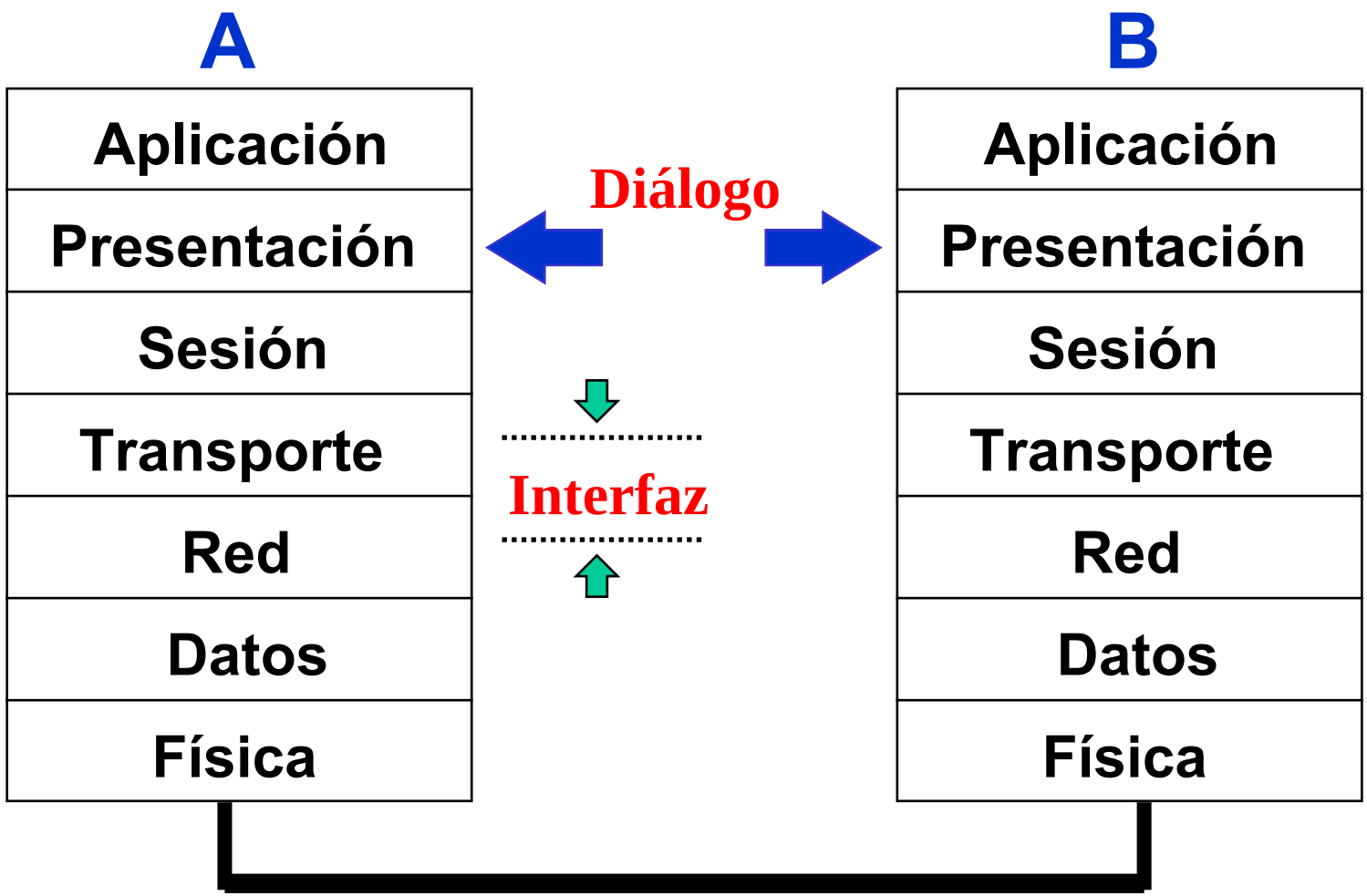
- *Transmisión simultánea en ambos sentidos.*
- *Puede ser por dos vías o por multiplexación de frecuencias.*



Protocolos

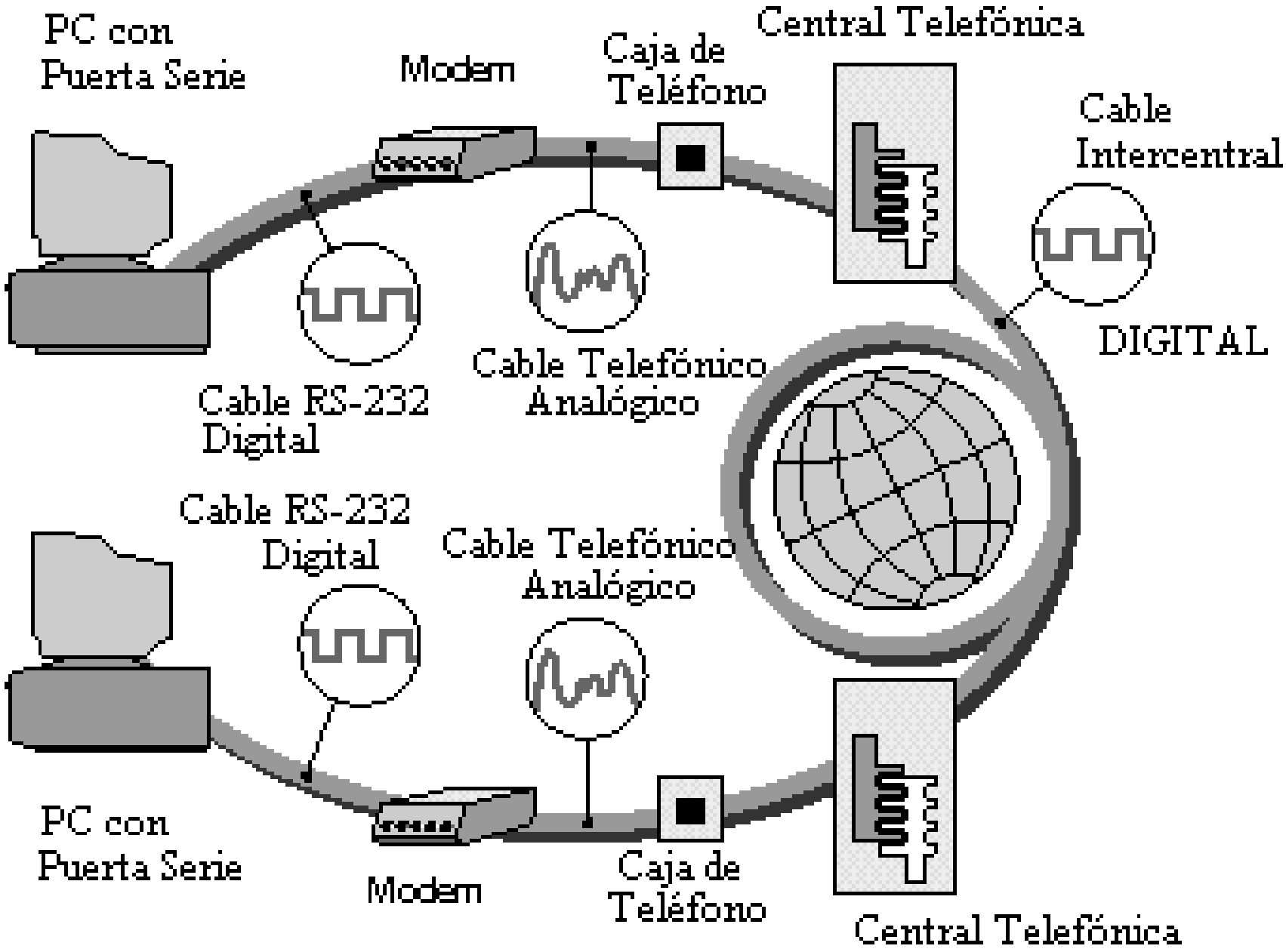
- *Son reglas de diálogo entre entidades.*
- *Nos permiten especificar interfaces.*
- *Se necesitan varios protocolos a distintos niveles para mantener una comunicación entre sistemas.*
- *Se tiende a la utilización de estándares en toda clase de comunicación.*

Modelo OSI de ISO



Interfaz RS-232

- *Comunica una computadora (DTE) con un modem (DCE).*
- *Define cables, conectores, señales, etc.*
- *Constituye los puertos serie de las PCs*
- *Se usa para disciplina de transferencia de datos, simplex y full duplex.*


PC con
Puerta Serie
Modem
Caja de
Teléfono
Central Telefónica
Cable
Intercentral
DIGITAL
Cable RS-232
Digital
Cable Telefónico
Analógico
Cable RS-232
Digital
Cable Telefónico
Analógico
PC con
Puerta Serie
Modem
Caja de
Teléfono
Central Telefónica

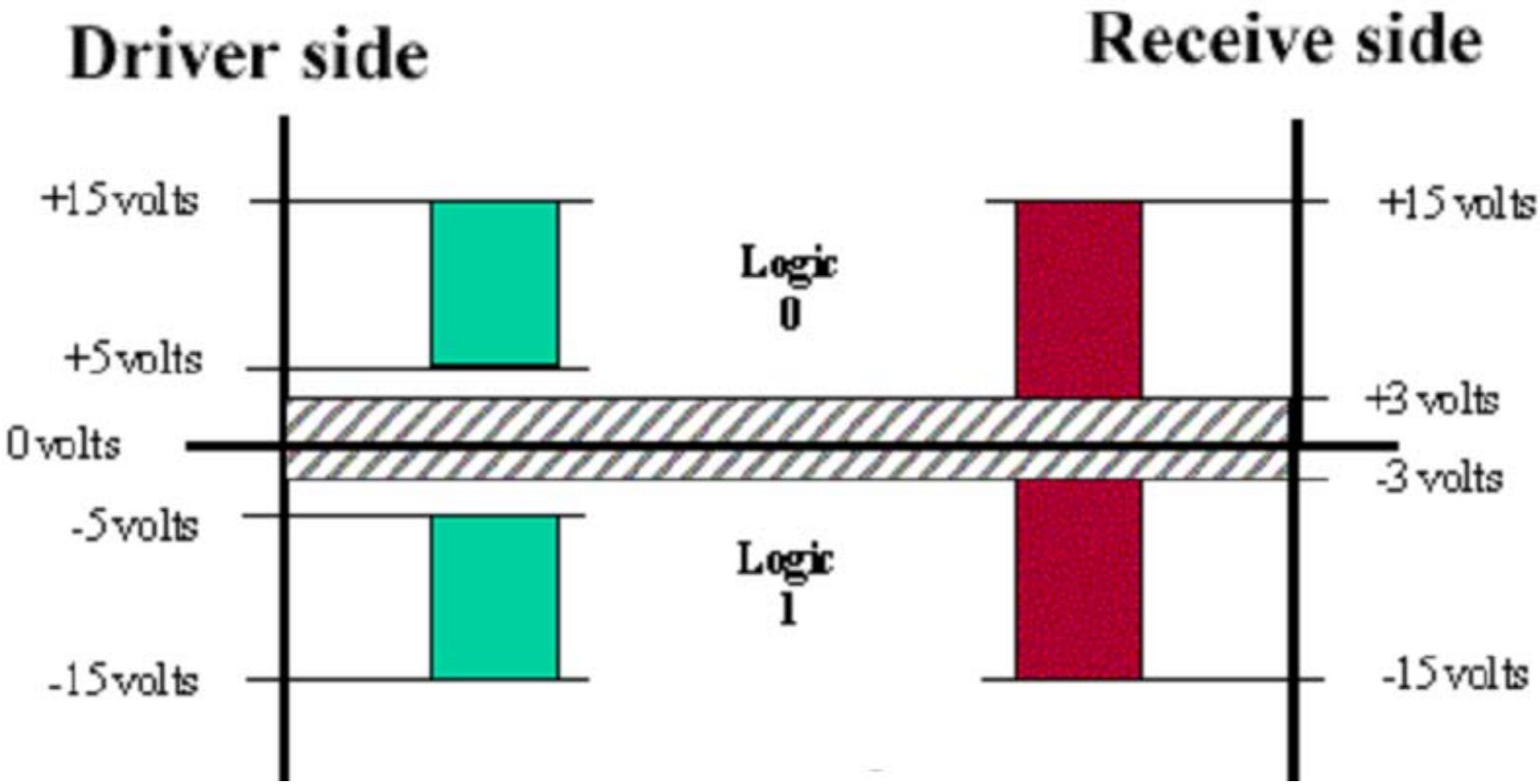
Conectores y Cables RS-232

- *Originalmente tiene 25 cables.*
- *Las PCs usan solo 9.*
- *Los conectores son DB25 y DB9*
- *Longitud máxima: 50 pies (15 metros)*
- *Señales: Marca y Espacio*

			Salida	Entrada
ESPACIO	0	(+)	5 a 15 V	3 a 15 V
MARCA	1	(-)	-5 a -15 V	-3 a -15 V

Data signals		
Level	Transmitter	Receiver
Logical 0	+5 V to +15 V	+3 V to +15 V
Logical 1	-5 V to -15 V	-3 V to -15 V
Undefine	-3 V to +3 V	

Control signals		
Signals	Driver	Terminator
"Off"	-5 V to -15 V	-3 V to -15 V
"On"	5 V to 15 V	3 V to 15 V



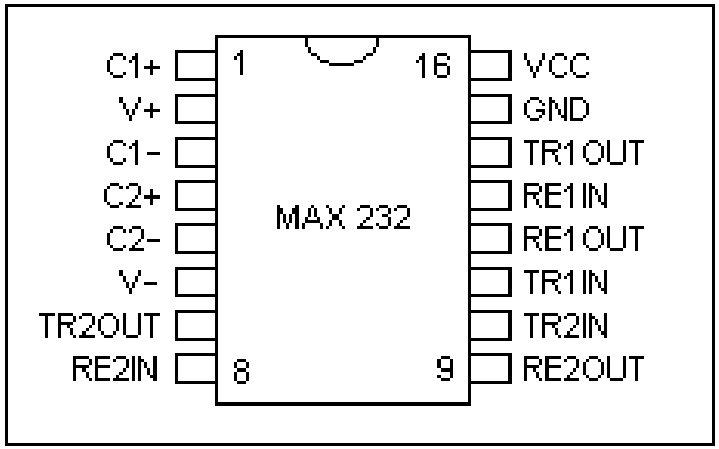
Señales Eléctricas RS-232

- *En líneas de datos el valor positivo es un cero y el valor negativo es un uno, esto se denomina lógica negativa.*
- *En líneas de control el valor positivo es habilitante y el valor negativo es deshabilitante. Para este caso tenemos lógica positiva.*

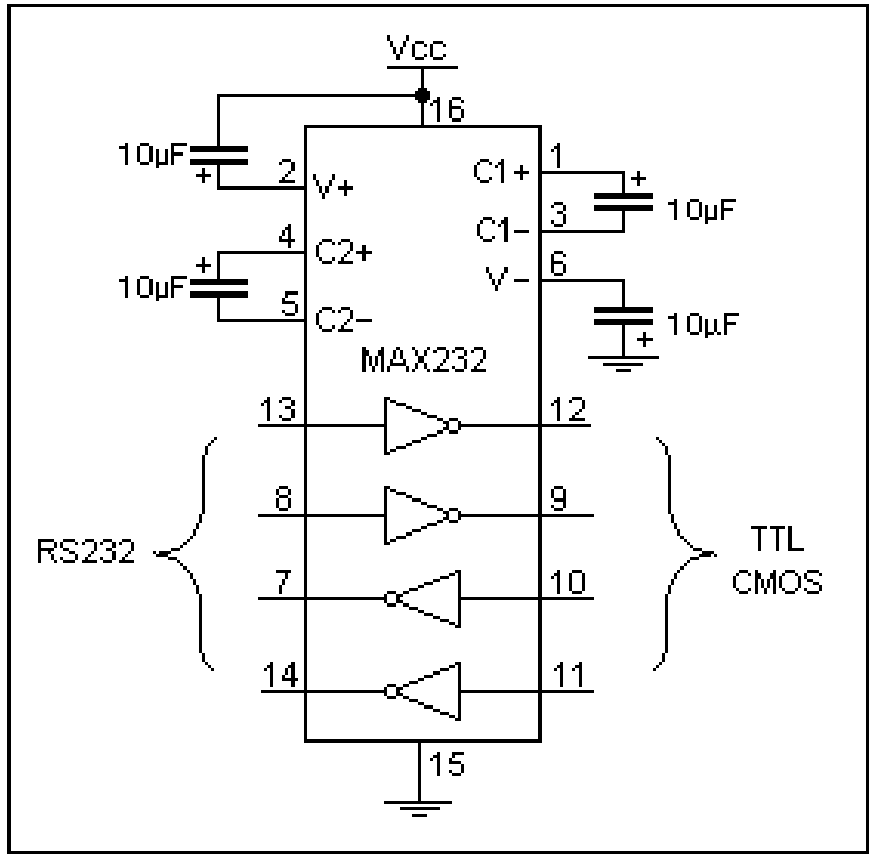
Convertidores TTL/RS-232

- *Para convertir señales RS-232 a TTL y viceversa se usa un integrado caracte-rístico, el MAX-232.*
- *Solamente con alimentación de 5 V. este circuito es capaz de generar seña-les con el nivel RS-232 y convertirlas en forma segura a TTL.*
- *Contiene dos líneas de transmisión y dos de recepción.*

MAX-232



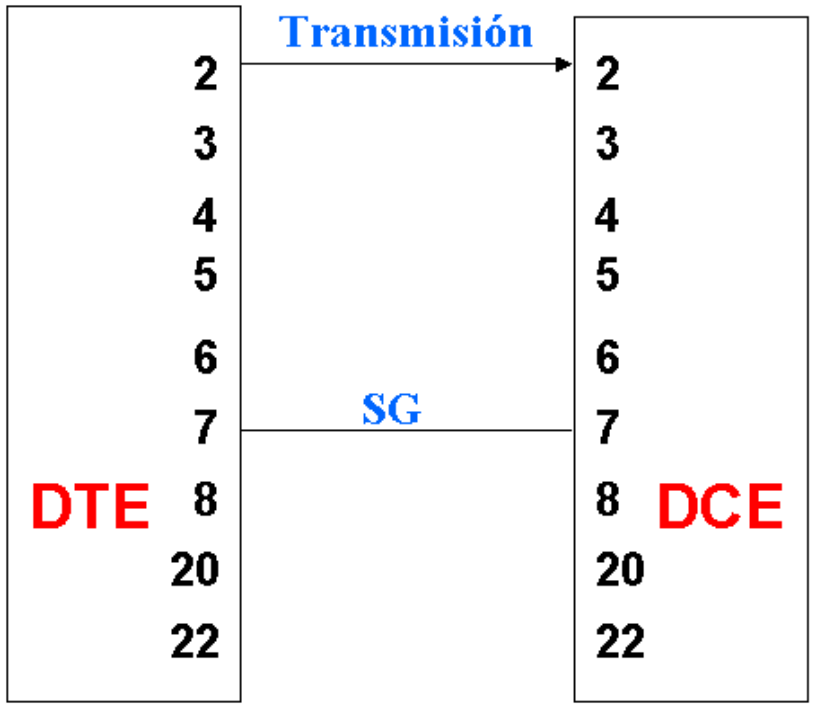
Pinouts for the MAX-232, RS-232 Driver/Receiver.



Typical MAX-232 Circuit.

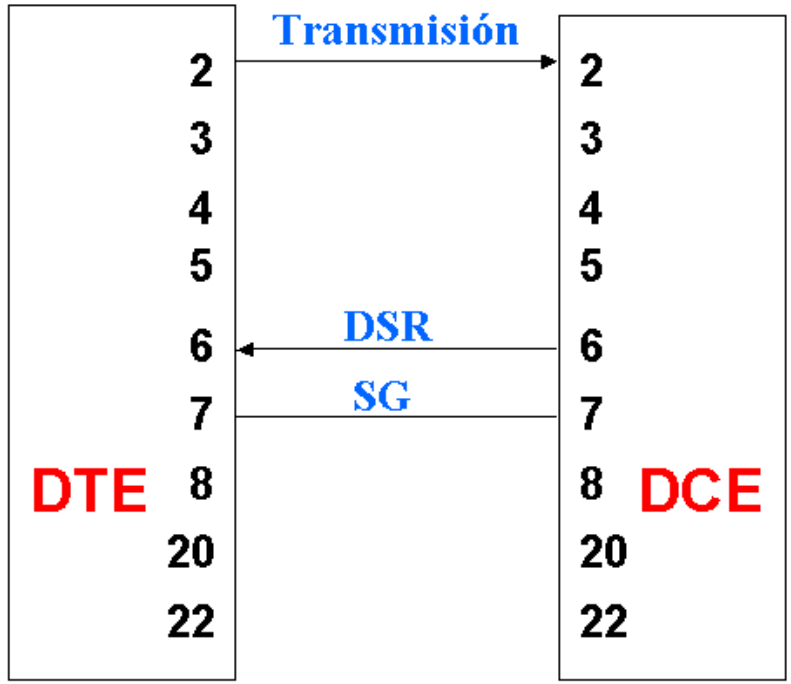
Comunicación Simplex

- *Desde un DTE a un DCE*



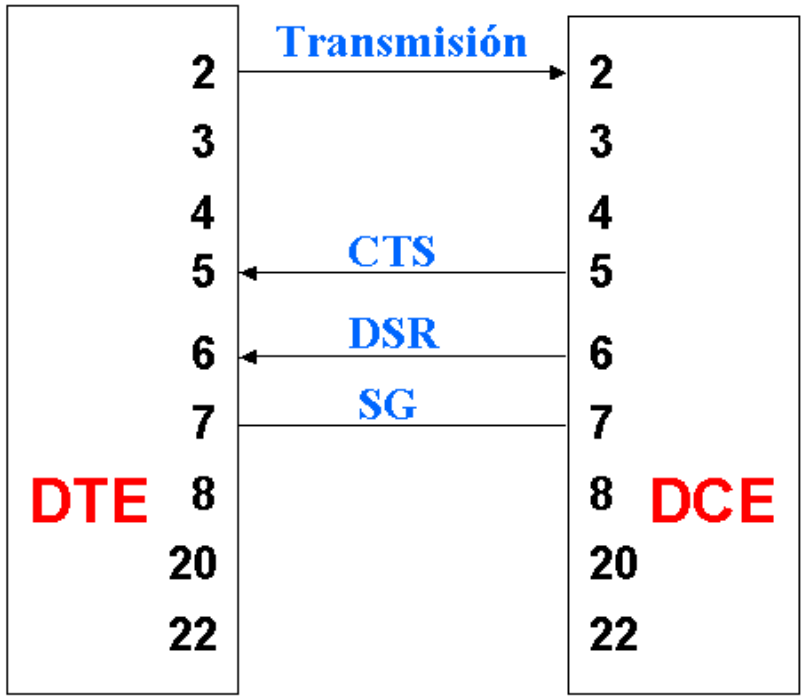
Transmisión Simplex

- *Con handshaking por hardware, si DSR es positivo es posible transmitir, NO en caso contrario.*



Transmisión Simplex

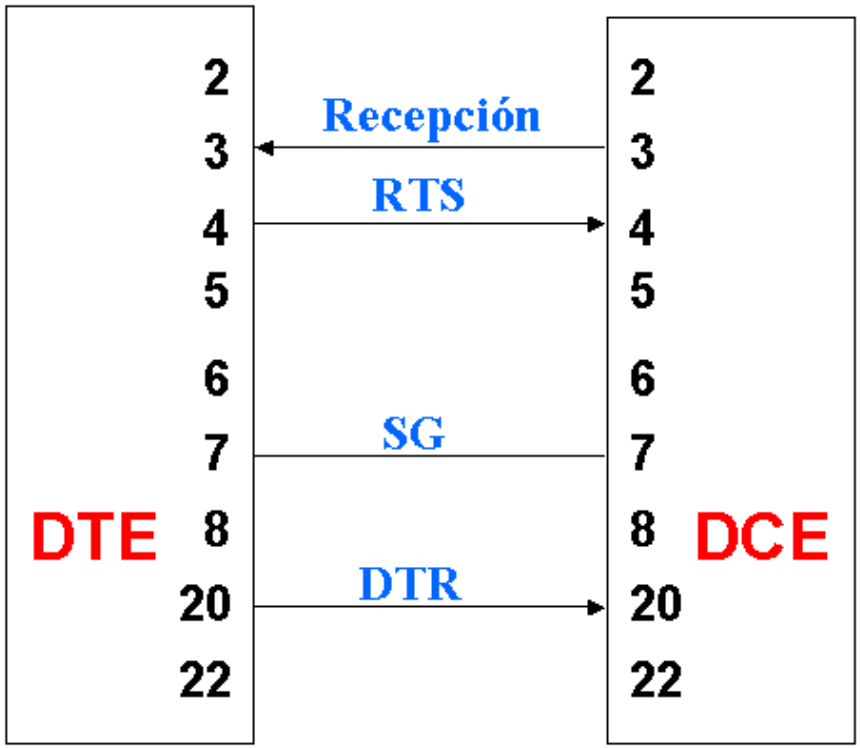
- *Incluyendo dos líneas de handshaking por hardware.*



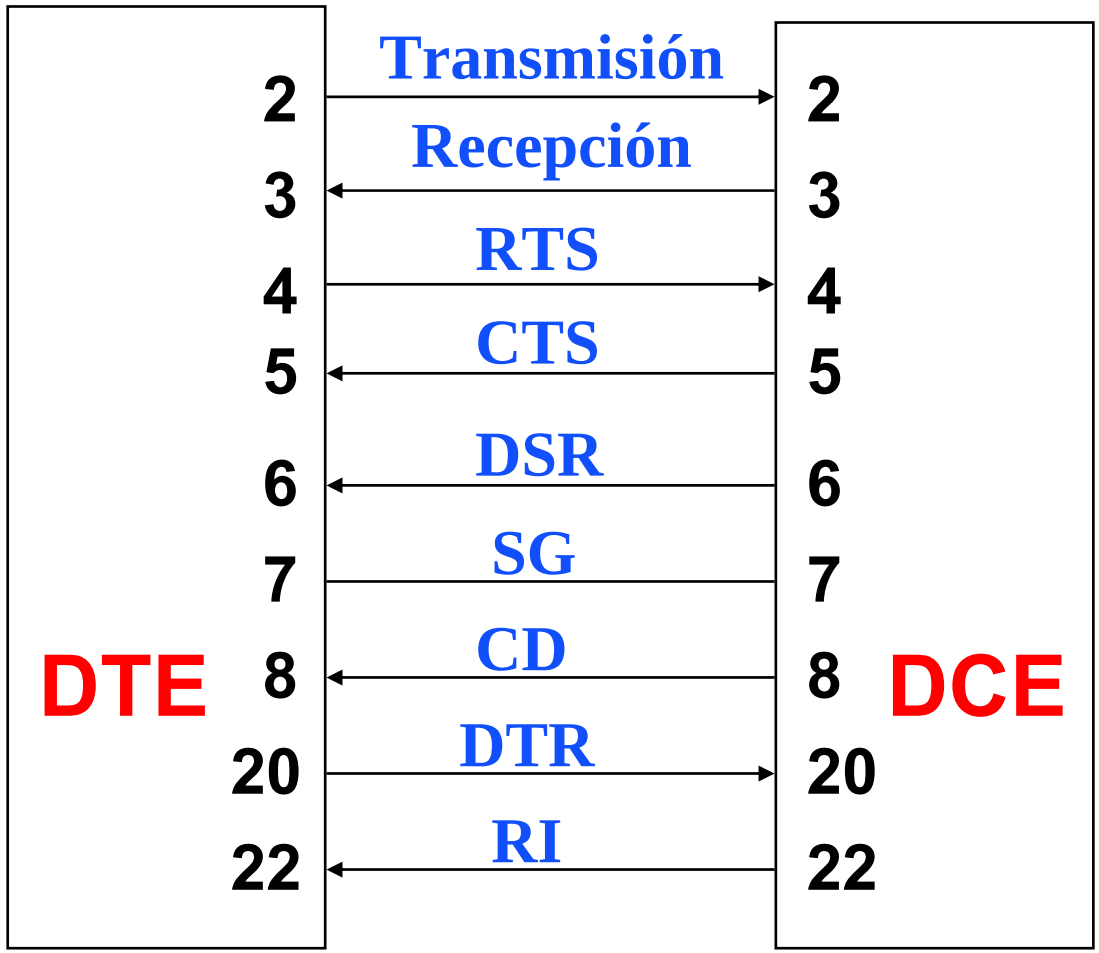
Se puede transmitir si CTS y DSR son positivos

Recepción Simplex

- *El handshaking es del DTE al DCE*



Comunicación DCE/DTE 2 vías



Cable Null Modem

- *El cable null modem se necesita cuando debemos conectar dos DTE entre si.*
- *Hay que cruzar la transmisión con la recepción y los respectivos hand-shakings.*
- *Esto permite que cada uno de los ex-tremos crea que tiene un DCE en el otro.*

Diagrama Null Modem

FORMA 1

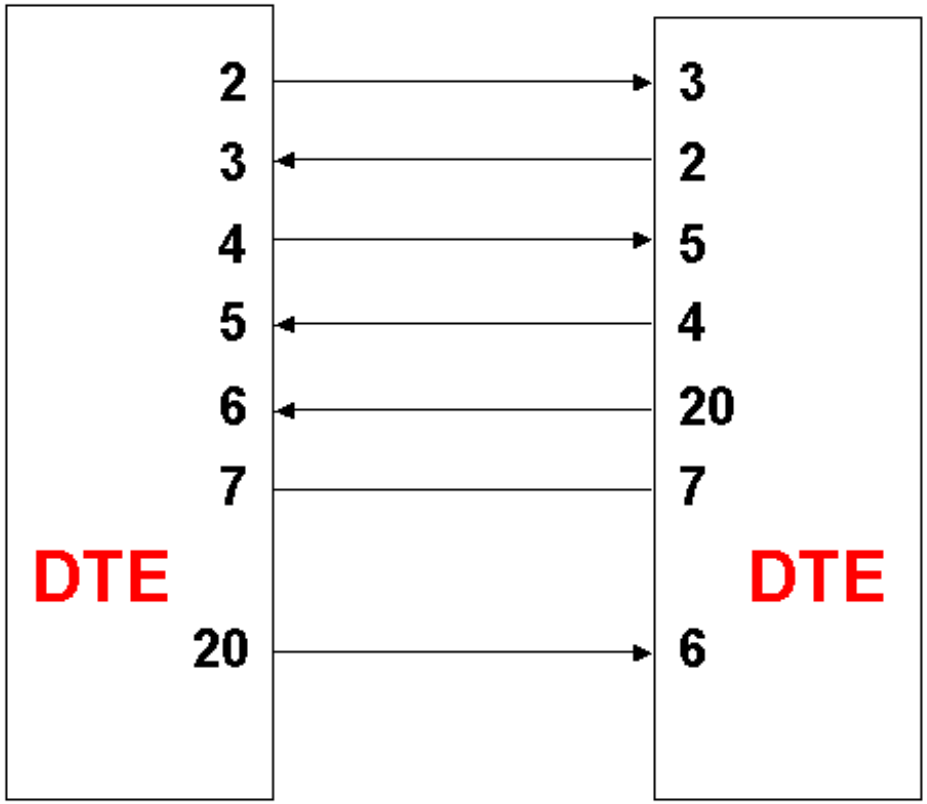


Diagrama Null Modem

FORMA 2

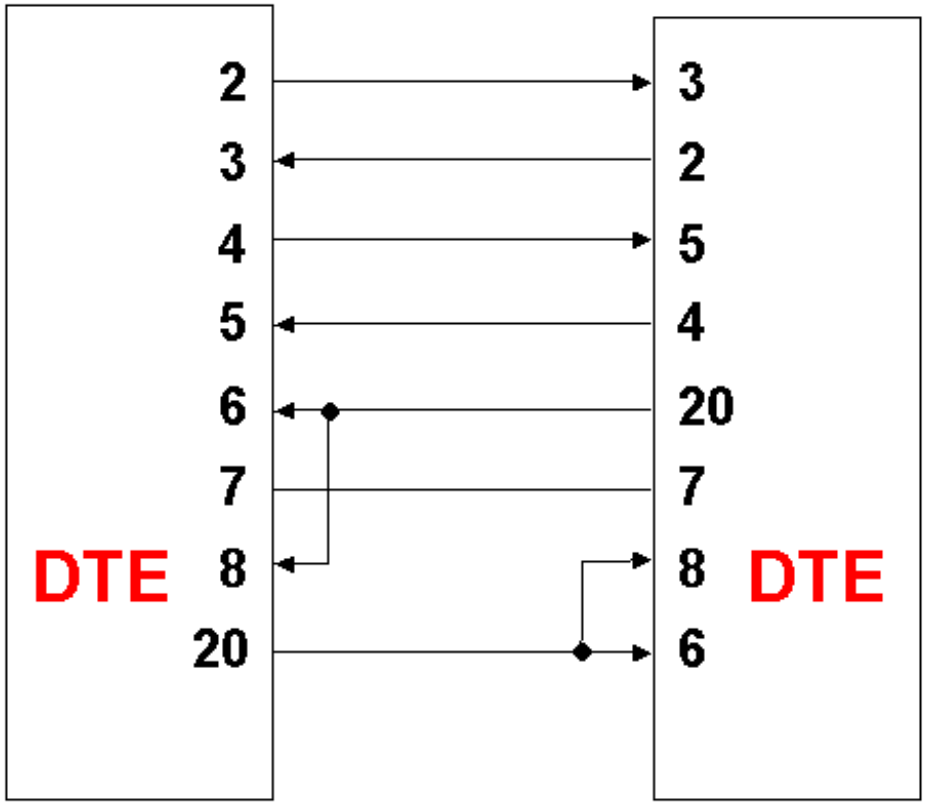
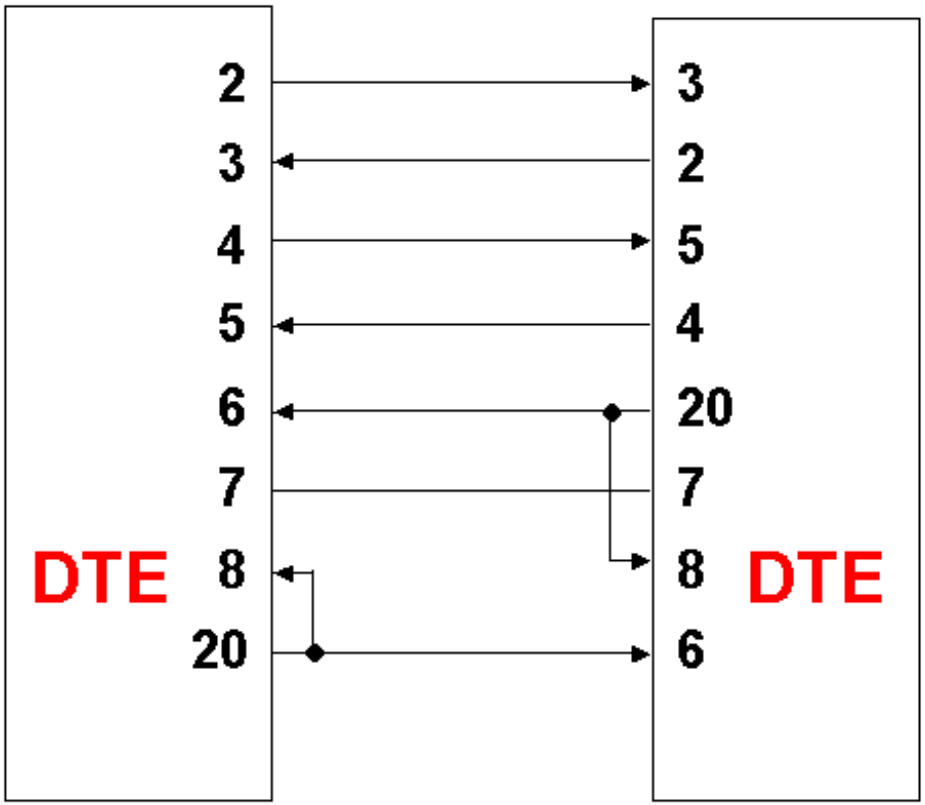


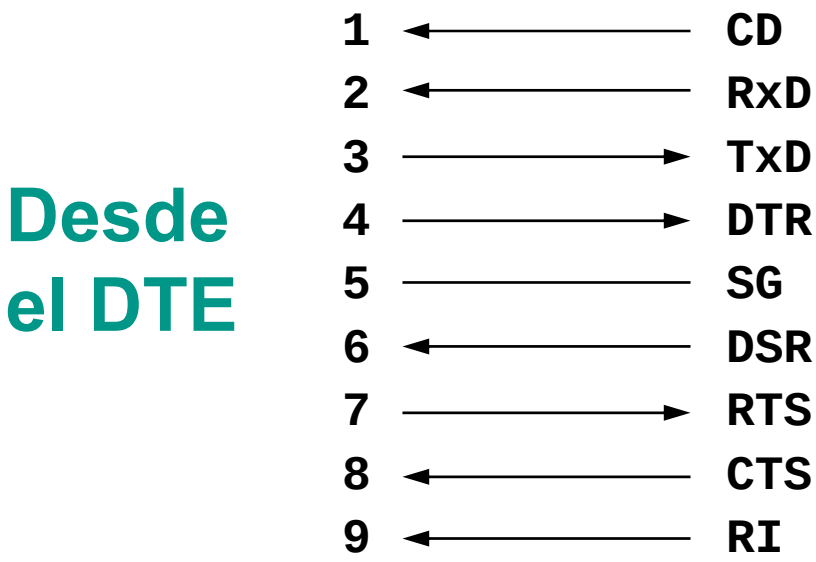
Diagrama Null Modem

FORMA 3

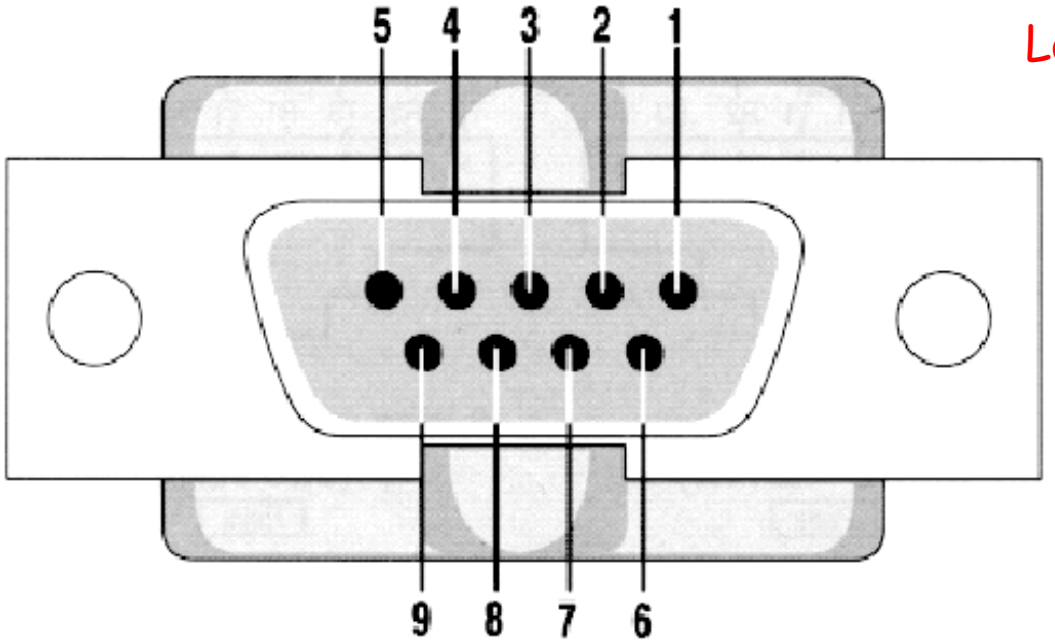


Conector DB9

- *Los números de pin indicados hasta ahora corresponden al conector DB25.*
- *Para el conector DB9 son:*



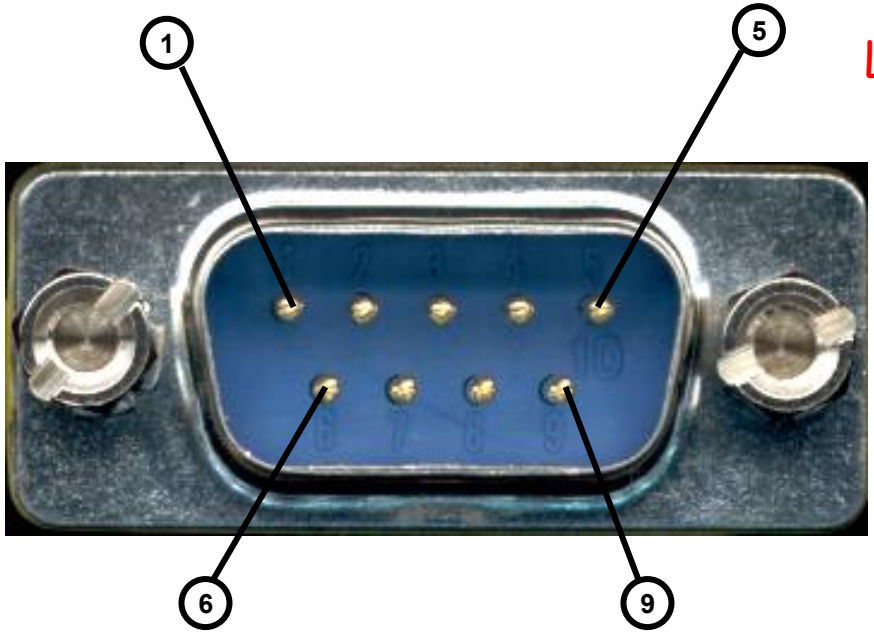
Conector DB-9



Lado cable

Pin	Signal	Pin	Signal
1	Data Carrier Detect	6	Data Set Ready
2	Received Data	7	Request to Send
3	Transmitted Data	8	Clear to Send
4	Data Terminal Ready	9	Ring Indicator
5	Signal Ground		

Conector DB-9



Lado PC

Pin	Signal	Pin	Signal
1	Data Carrier Detect	6	Data Set Ready
2	Received Data	7	Request to Send
3	Transmitted Data	8	Clear to Send
4	Data Terminal Ready	9	Ring Indicator
5	Signal Ground		

Tabla de longitud vs velocidad

- *Tabla de longitudes vs. veloc. de transm.*

Velocidad	***Pies***	***metros***
2400	***200***	***60***
4800	***100***	***30***
9600	***50***	***15***
19200	***25***	***7.6***
38400	***12***	***3.7***
56000	***8.6***	***2.6***

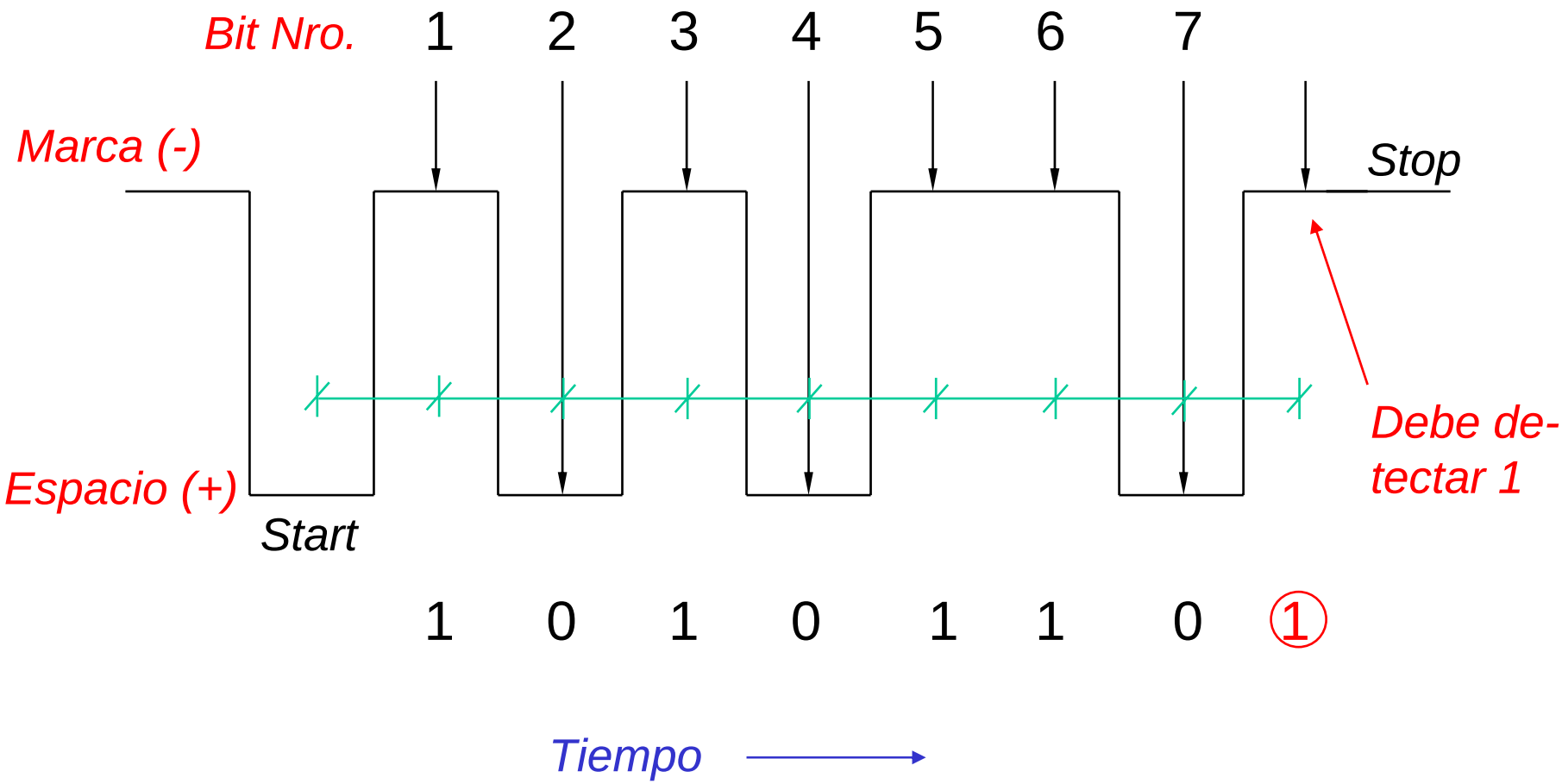
Transmisión de Caracteres

- *Los caracteres que en la computadora circulan en paralelo, de a uno o más bytes, deben ser transformados en una cadena de bits para ser transmitidos por un puerto serie.*
- *A diferencia del puerto paralelo, en el puerto serie los caracteres viajan de un bit por vez, los que son encolumnados en forma asíncrona.*

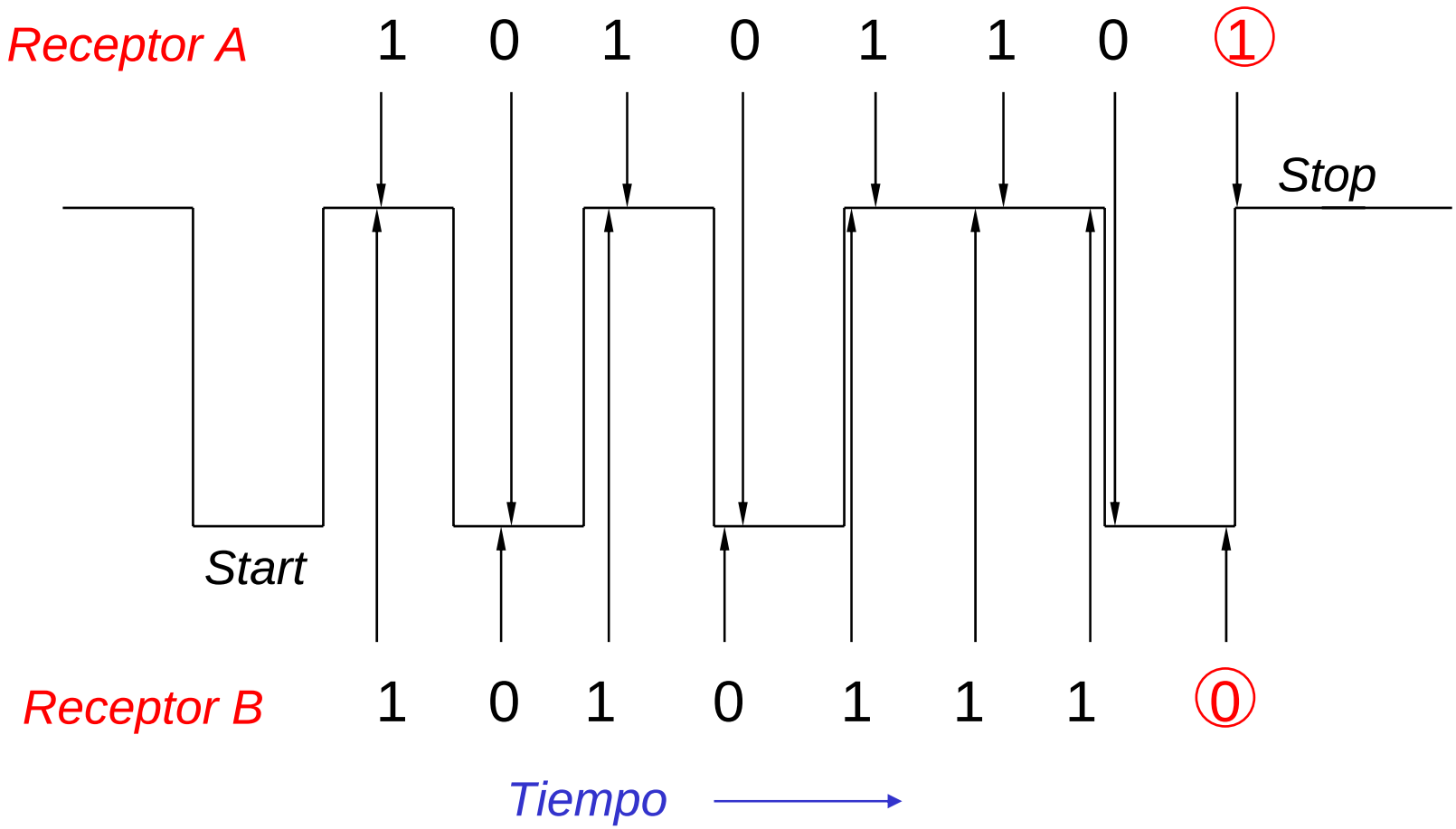
Transmisión Síncrona y Asíncrona

- *Síncrona: Se transmite un grupo de bytes (mensaje) en forma secuencial, después del bloque de sincronización. En general es más rápida que la asíncrona. Requiere calidad en el control de temporización.*
- *Asíncrona: Sincroniza byte por byte, es la que se utiliza mayoritariamente en las PCs. No es crítica en temporización. Si esta falla se produce un error de trama.*

Descripción de una Trama



Error de Trama



Descripción de una Trama

- *La unidad transmitida en comunicaciones asíncronas se llama trama, y la construcción de esta: tramado.*
- *Una trama tiene:*

- Bit de arranque

- Bits de datos

- Paridad

- Bit(s) de parada

Elementos de una Trama

- *Bit de arranque*
- *Bits de datos* — *5, 6, 7, 8*
- *Paridad* — *N, O, E, M, S*
- *Bits de Parada* — *1, 1.5, 2*

8N1

Descripción de una Trama

- *La línea antes de la transmisión esta en MARK o (-) (1 lógico).*
- ***Bit de arranque**: Se produce cuando aparece un carácter SPACE o voltaje (+) (0 lógico)*
- ***Bits de datos** : Va de 5 a 8 pero lo más común es 7 u 8. Viajan desde el bit menos al más significativo.*

Descripción de una Trama

- ***Paridad**:*
  - *Even (Par)*
  - *Odd (Impar)*
  - *No Parity (Sin paridad)*
  - *Space (Siempre cero)*
  - *Mark (Siempre uno)*

- ***Bits de parada**: Pueden ser 1, 1.5 ó 2 bits, que indican el final del carácter.*

Definiciones

- ***Break**: Es bajar la línea por más tiempo que la máxima cantidad de **espacios** acumulables, pretende llamar la atención del sistema (p.ej. cuando está tildado).*
- ***Baudios**: Son la medida de señalización eléctrica fundamental en la línea. Si la onda posee dos estados la cantidad de bits por segundo es igual a la de baudios.*

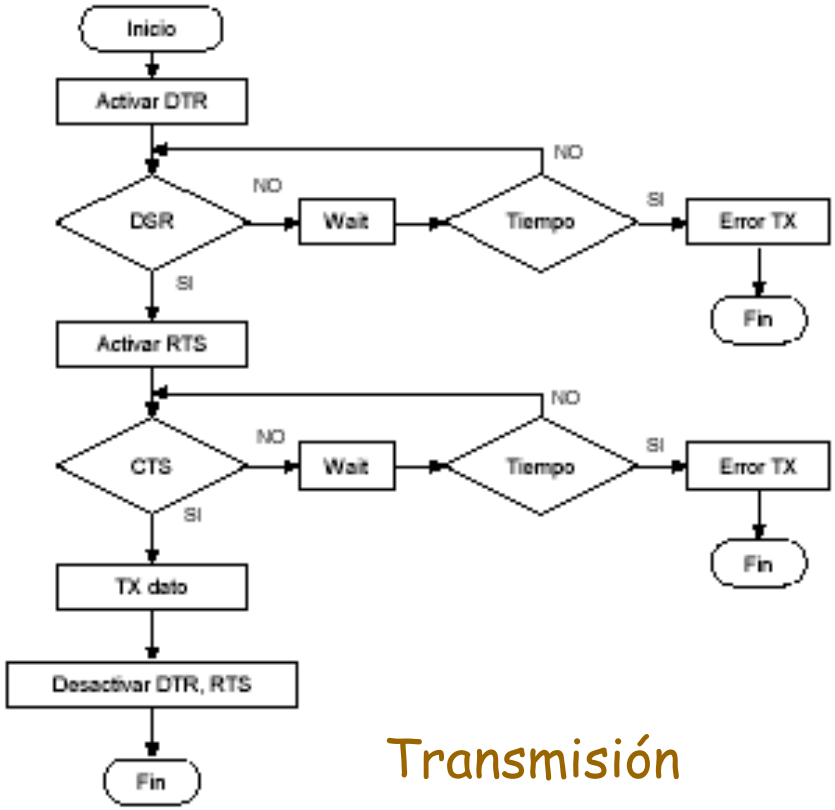
Reconocimientos

- *Tenemos reconocimientos (handshaking) por hardware y por software.*
- ***Hardware:** Los **DCE** usan **DSR** (Data Set Ready) como línea principal de hand-shaking, **CTS** (Clear To Send) es la línea secundaria.*
- *Los **DTE** usan **DTR** como línea principal y **RTS** como línea secundaria.*

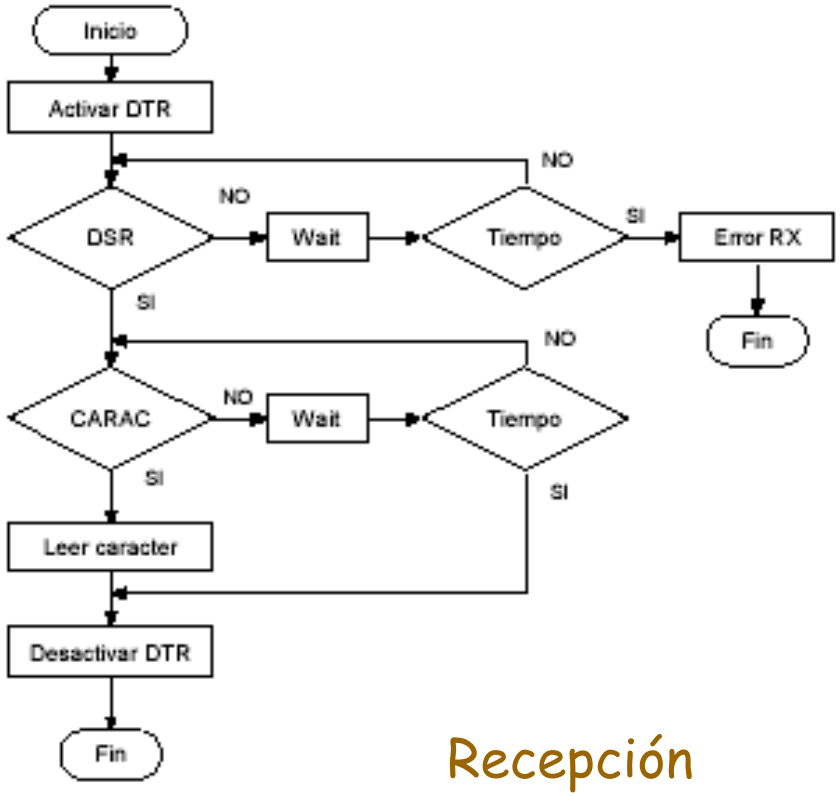
```
Start
do sinfin
  do while NOT Carácter para enviar
  end-do
  reset contador timeout
  while NOT timeout do
    if DSR alto
      if CTS alto
        Enviar carácter
        if EOF
          Levanta bandera EOF
          return( OK )
        end-if
      else incrementa contador timeout
      end-if
    else incrementa contador timeout
    end-if
  end-while
  if timeout > maximo
    return( error de timeout )
  end-if
end-do
End
```

Seudocódigo de reconocimientos por hardware

Diálogo - Diagrama de Flujo



Transmisión



Recepción

Manejo del puerto por BIOS

- *Para acceder a estos servicios a través de programas de aplicación se puede utilizar el BIOS.*
- *Esta vía tiene ciertas limitaciones por lo que si se desea control total conviene manejar directamente la UART.*
- *UART: Universal Asynchronous Receiver Transmiter. Es el chip del puerto serie. Cada puerto serie es una UART.*

Manejo del puerto por BIOS

- *En Borland C el servicio disponible es:*

```
int bioscom(int cmd, char abyte, int port)
```

- *Está en la biblioteca <bios.h>*
- ***cmd** Tiene las siguiente posibilidades:*

```
0   Inicializar el puerto
1   Enviar un carácter
2   Recibir un carácter
3   Devolver el estado del puerto
```

Manejo del puerto por BIOS

- ***abyte*** *Para cmd=1 ó cmd= 2.*
  Es el byte a enviar o recibir.
  Para cmd=0
  Es el byte de inicialización.
- ***port*** *Tiene la siguiente configuración:*

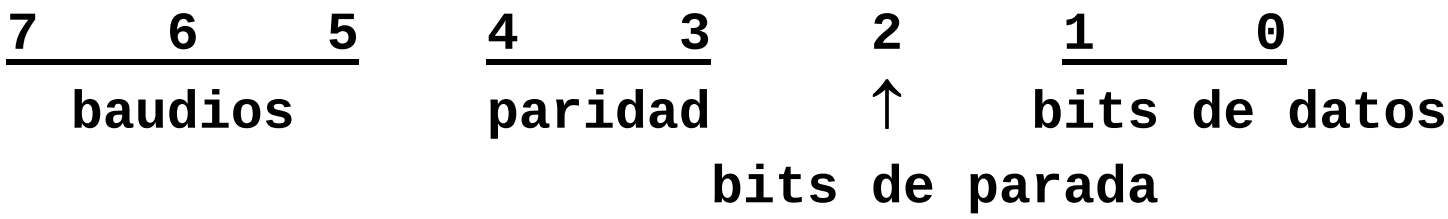
0	**COM1:**
1	**COM2:**
2	**COM3:**
3	**COM4:**

Inicialización del Puerto Serie

- *El comando de inicialización para el puerto COM1 es el siguiente:*

```
bioscom( 0, abyte, 0 );
```

- *abyte tiene la siguiente configuración:*



Inicialización del Puerto Serie

- ***baudios*** *tiene el siguiente significado:*

9600	111
4800	110
2400	101
1200	100
600	011
300	010
150	001
110	000

- ***paridad***

Sin	00 ó 10
Impar	01
Par	11

Inicialización del Puerto Serie

- ***bits de parada***

uno	**0**
dos	**1**

- ***bits de datos***

siete	**10**
ocho	**11**

(las otras combinaciones no son válidas)

Palabra de Inicialización

- *abyte = OR bit a bit de lo siguiente:*

```
0x02 = 7 data bits      0x00 = 1 stop bit
0x03 = 8 data bits      0x04 = 2 stop bits

0x00 == no parity
0x08 == odd parity
0x18 == even parity

0x00      110 bps       0x20   150 bps
0x40      300 bps       0x60   600 bps
0x80     1200 bps       0xa0  2400 bps
0xc0     4800 bps       0xe0  9600 bps
```

Palabra de Inicialización

- *Ejemplo:*

abyte = 0x8B = (0x80 | 0x08 | 0x00 | 0x03)

El puerto se configura a:

1200 baud	**0x80**
Paridad impar	**0x08**
1 bit parada	**0x00**
8 bits datos	**0x03**
	0x8B

Función en C de Microsoft

```
unsigned _bios_serialcom( unsigned cmd,
                          unsigned puerto, unsigned data);
```

- *Cambia el nombre de los parámetros y su ubicación pero son los mismos que los vistos en el caso anterior.*

Puertos Serie en una PC

- *Controlados por una UART*

8250

16450

16550

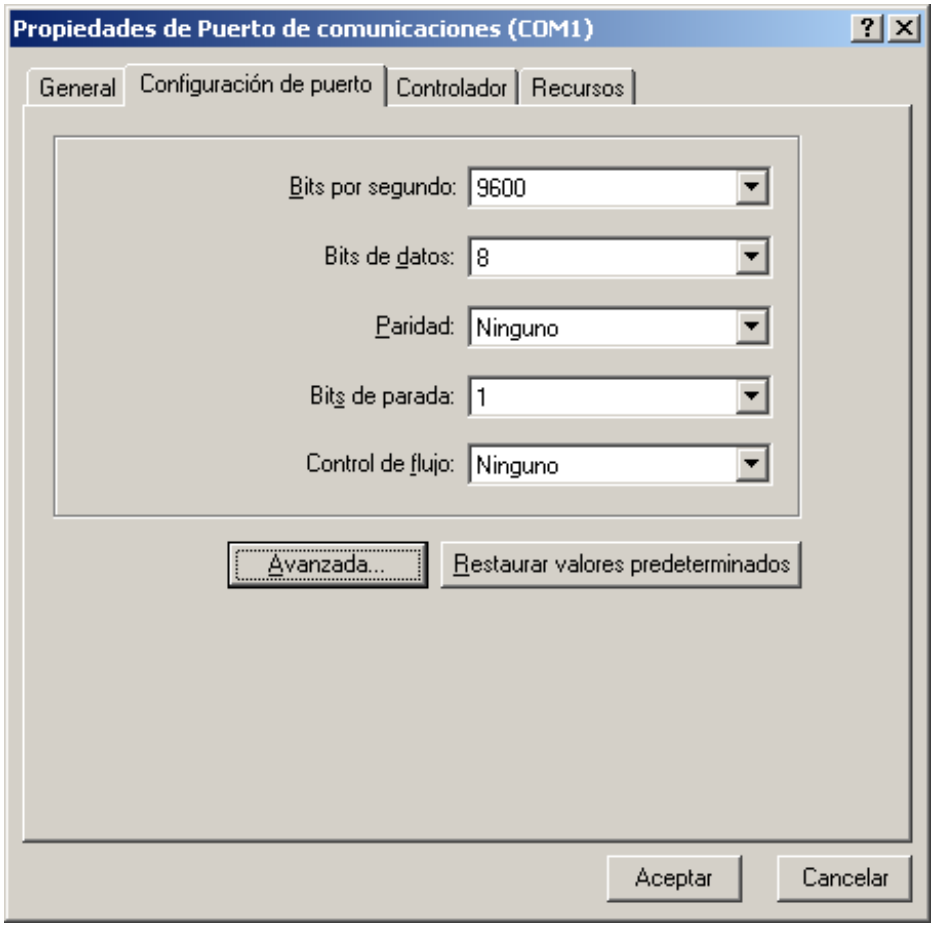
- *Hoy lo más común es 16550, también están disponibles las 16650 y 16750.*

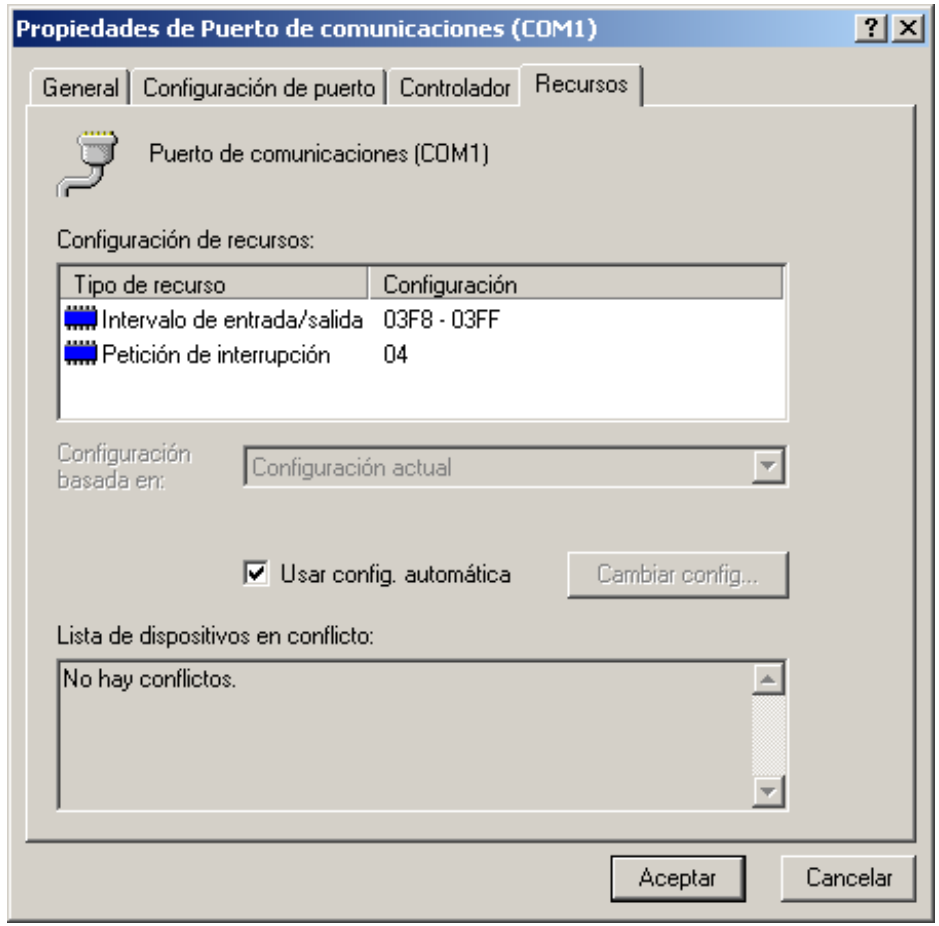
Direcciones de Puerto Serie

- *Por defecto podemos llegar a tener cuatro puertos serie (aunque solo dos interrupciones)*

Puerto	Dirección	Interrupción
COM1	0x3F8	4
COM2	0x2F8	3
COM3	0x3E8	4
COM4	0x2E8	3

Desde Panel de Control - Windows



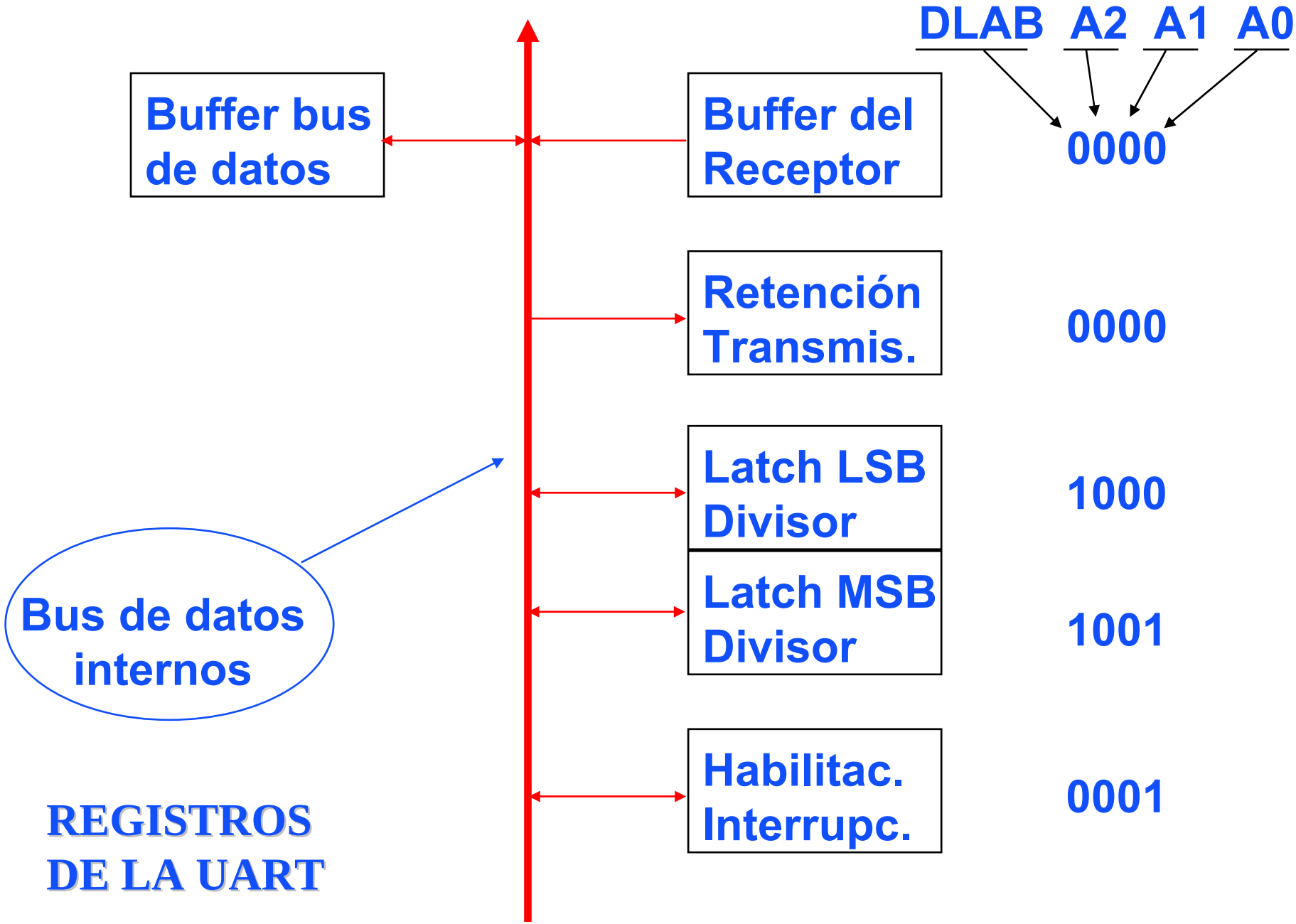


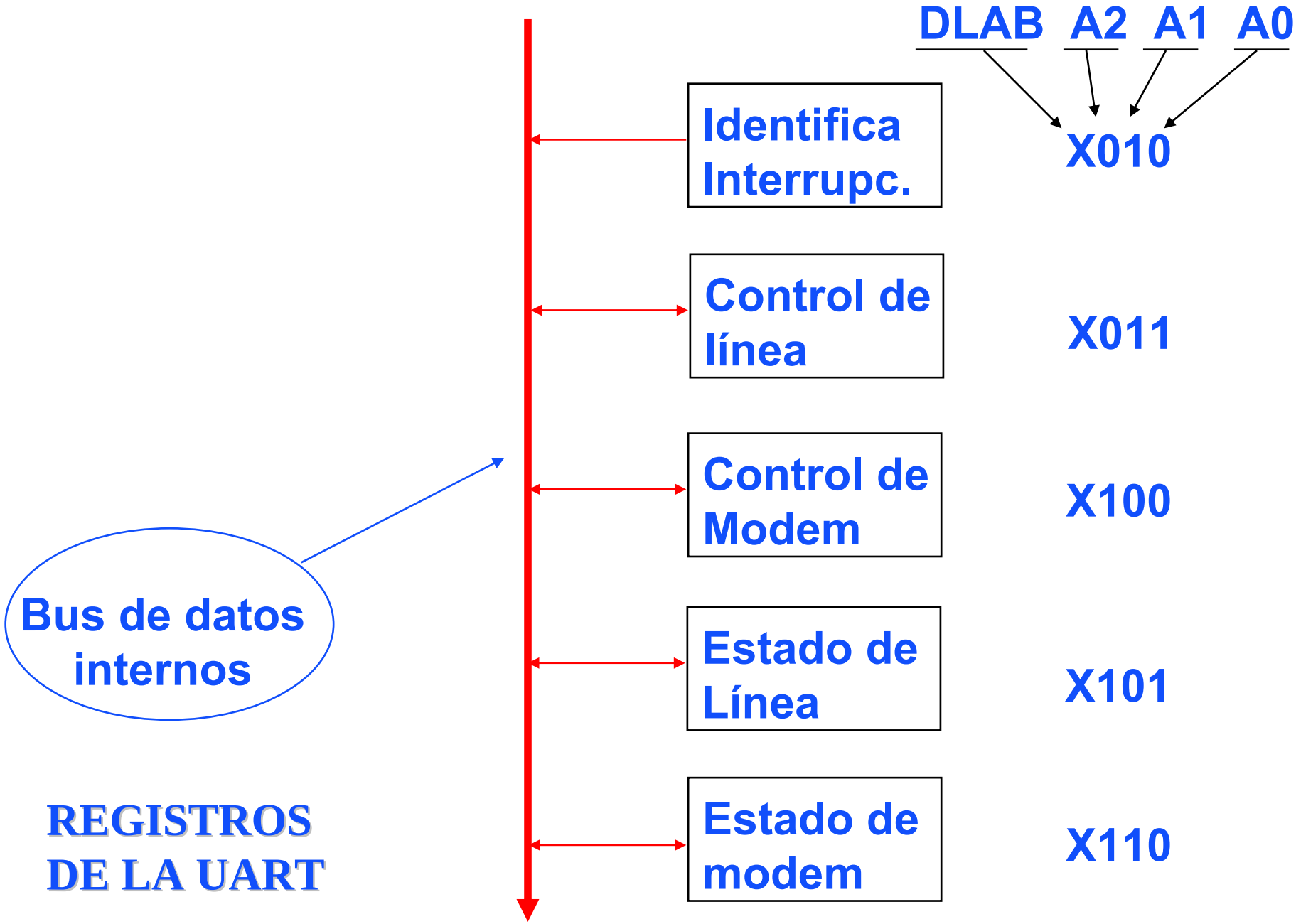
Registros de la UART

- Control: Recibe comandos de la CPU
- Estado: Informa a la CPU el estado de la UART
- Buffer: Mantiene caracteres entrantes y salientes en la transmisión

Registros de la UART

Control	**Line Control Register** **Modem Control Register** **Interrupt Enable Register** **Baud Rate Divisor Latches**
Estado	**Line Status Register** **Modem Status Register** **Interrupt Identification Reg.**
Buffer	**Receive Buffer Register** **Transmitter Holding Register**

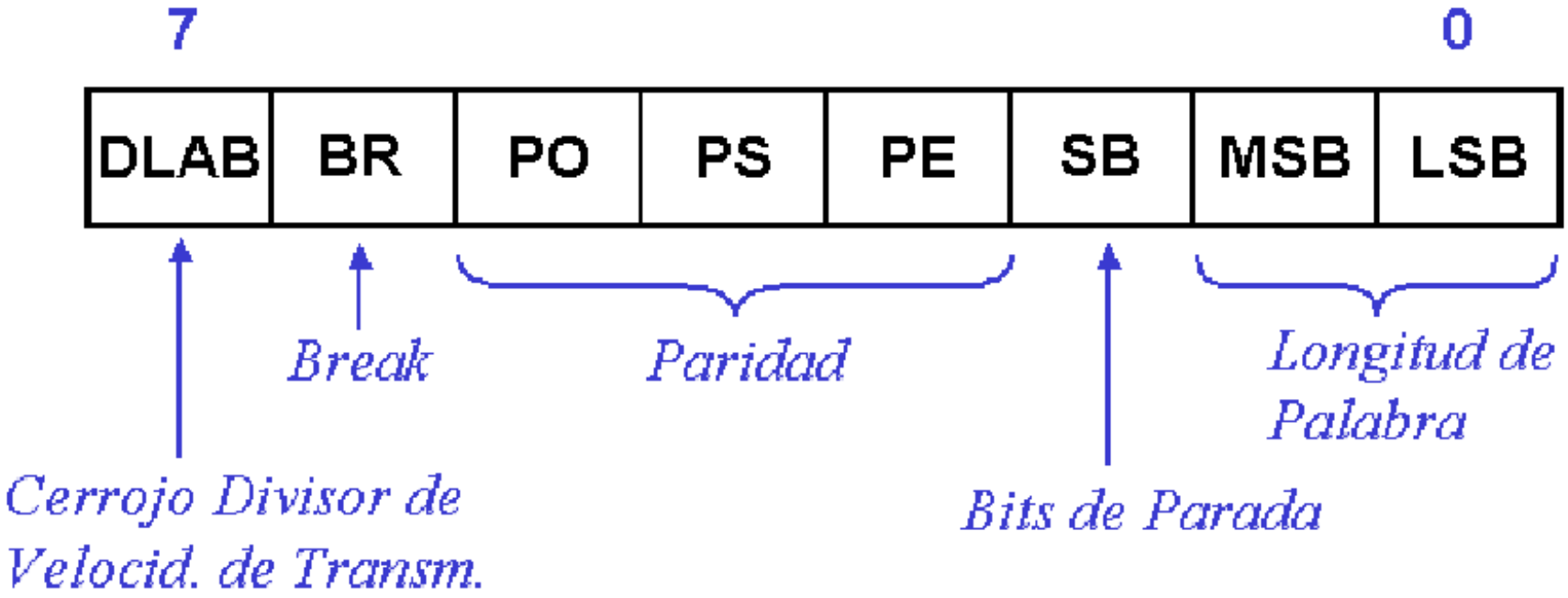

DLAB A2 A1 A0
Buffer bus de datos
Buffer del Receptor
0000
Retención Transmis.
0000
Latch LSB Divisor
1000
Latch MSB Divisor
1001
Habilitac. Interrupc.
0001
Bus de datos internos
REGISTROS DE LA UART



REGISTROS DE LA UART

Registro de Control de Línea

- *Line Control Register: Se usa para especificar los parámetros de comunicaciones.*



Registros de Control

- *Line Control Register:* Se usa para especificar los parámetros de comunicaciones.

Bit			
0	Word length LSB	00 ➡ 5	01 ➡ 6
1	Word length MSB	10 ➡ 7	11 ➡ 8
2	Stop bits	0 ➡ 1	1 ➡ 2
3	Parity enable	000 ➡ ninguna	101 ➡ mark
4	Parity select	001 ➡ impar	111 ➡ space
5	Parity one	011 ➡ par	
6	Break	0 ➡ off	1 ➡ on
7	Divisor Latch Access Bit (DLAB)		

Registro de Control de Línea

- ***Cerrojo Divisor de Velocidad de Transmisión (Baud Rate Divisor Latch):*** *Permite el acceso al divisor de la frecuencia fija de reloj para seleccionar la velocidad de transmisión.*

- ***Break****: Cuando este bit se pone en 1 se coloca la señal de break en la línea de transmisión.*

Temporización de la UART

- *La frecuencia del reloj de temporización de la UART es de 1.8432 MHz.*
- *Esta frecuencia se divide en forma fija por 16, lo que determina la velocidad máxima de transmisión de 115200 bps.*
- *Mediante dos registros se controla el divisor de esta frecuencia que nos permite establecer las distintas velocidades de transmisión.*

Temporización de la UART

- *Uno de los registros es el DLL (Divisor Latch Less significant byte) y el otro es el DLM (Divisor Latch Most significant byte)*
- *Para velocidades que superan los 300 bps ya no se utiliza el registro más significativo, por lo que en la actualidad sólo se configura el registro DLL.*
- *La tabla siguiente muestra: valor del divisor versus velocidad de transmisión.*

Temporización de la UART

Baud Rate	Decimal	Hex	MSB	LSB
300	384	180	1	80
1200	96	60	0	60
2400	48	30	0	30
4800	24	18	0	18
9600	12	0C	0	0C
14400	8	08	0	08
19200	6	06	0	06
38400	3	03	0	03
57600	2	02	0	02

Registro de Control de Línea

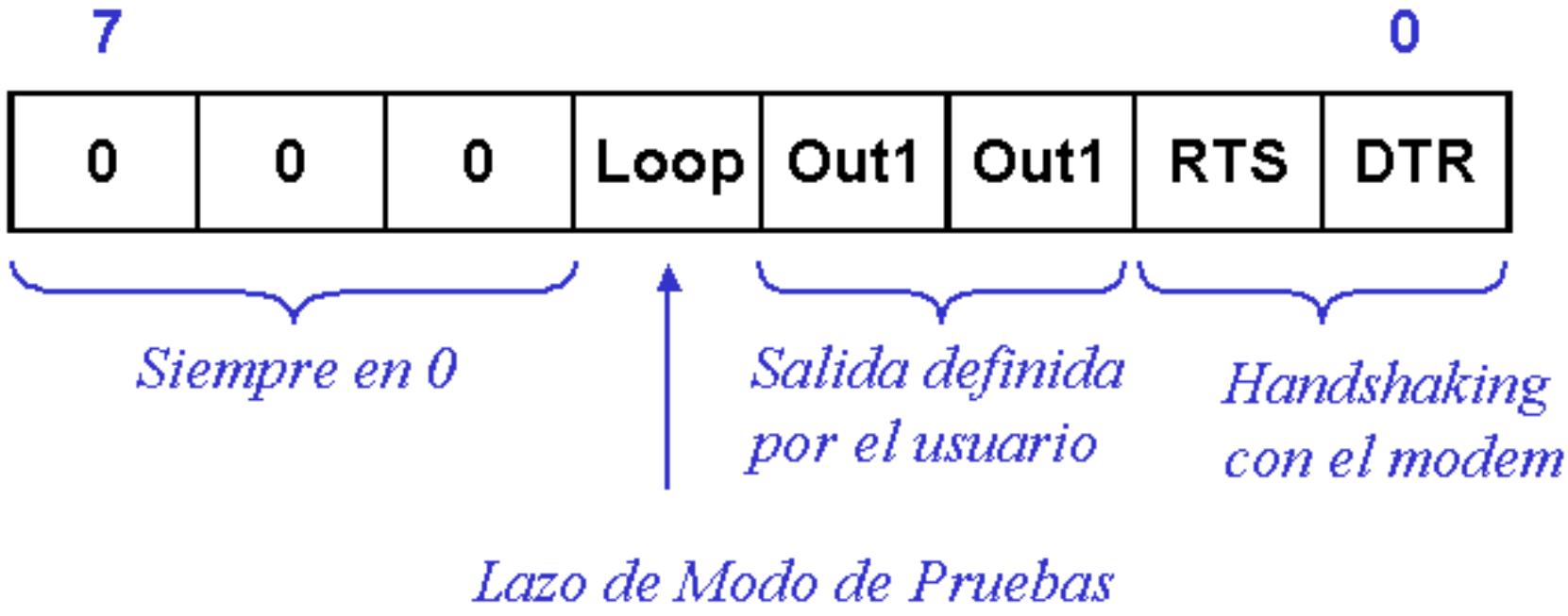
- ***Paridad:*** *Son tres bits que definen las cinco combinaciones posibles de paridad. El más significativo (Parity One) está en 1 cuando la paridad es fija (Mark o Space), el que le sigue (Parity Select) indica la clase de paridad seleccionada, el menos significativo (Parity Enable) está en 1 cuando se adopta alguna clase de paridad.*
- ***Bits de Parada:*** *Si está en 0 hay un solo bit de parada, si está en 1 tenemos 2.*

Registro de Control de Línea

- ***Longitud de Palabra:*** *Son dos bits que permiten seleccionar una longitud de 5, 6, 7 u 8 bits, los más comunes son 7 y 8.*
- *Al inicializar un puerto serie DEBE configurarse el Registro de Control de Línea, caso contrario es imposible transmitir.*
- *Una vez seleccionada la velocidad de transmisión debe tenerse cuidado en colocar el bit DLAB en cero para continuar con la operación normal.*

Registro de Control de Modem

- *Modem Control Register: Controla los re-conocimientos con el modem.*

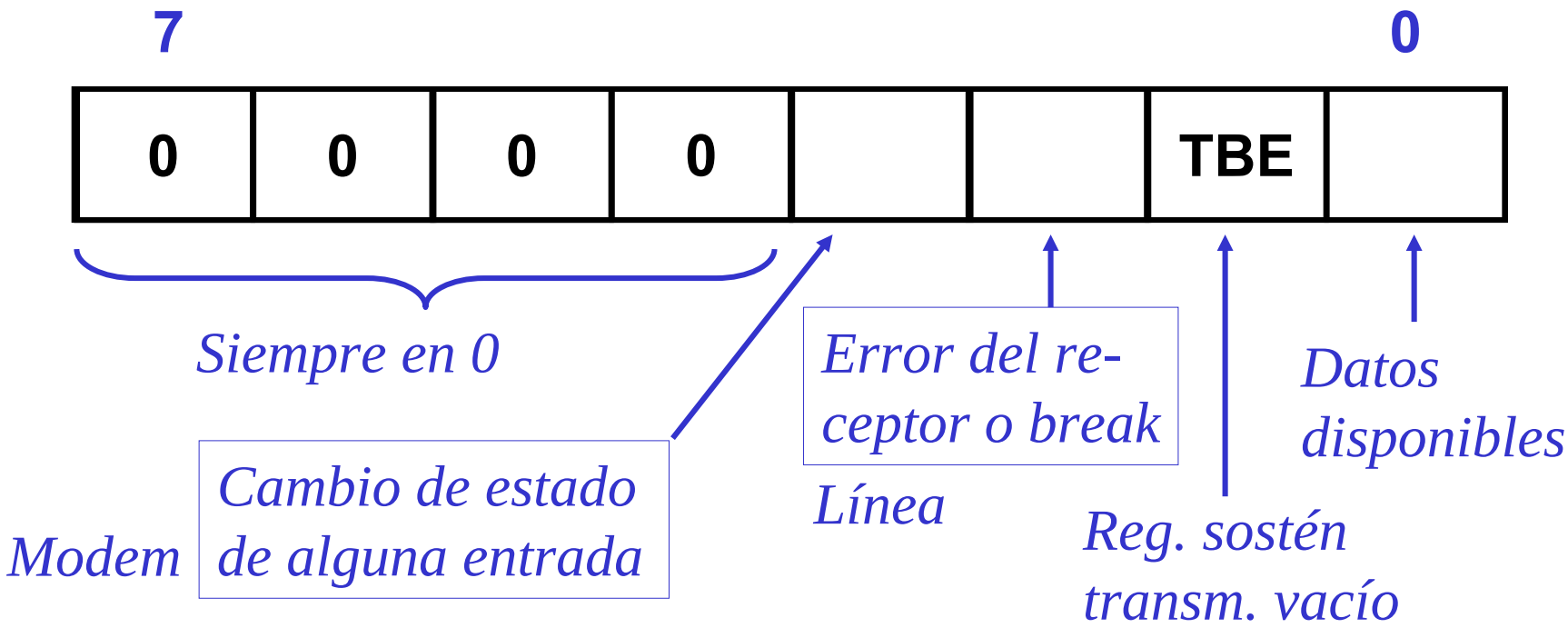


Registro de Control de Modem

- ***Loop:*** *Se utiliza para autoverificación.*
- ***Out1 y Out2:*** *Son salidas que no se reflejan en el conector del puerto serie.*
- ***RTS:*** *Request To Send, es el reconocimiento secundario desde el DTE.*
- ***DTR:*** *Data Terminal Ready, es el reconocimiento primario del DTE. Indica que el equipo está listo.*

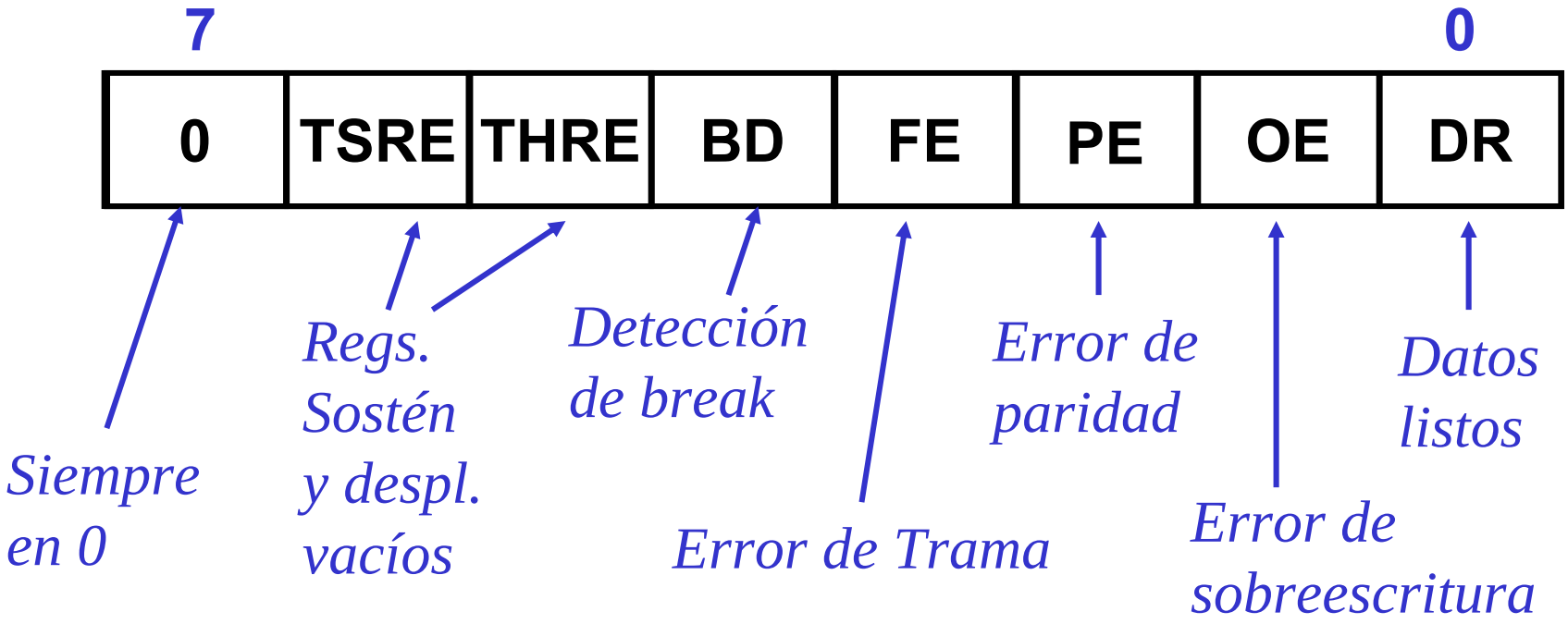
Registro de Habilitación de Interr.

- *Interrupt Enable Register: Si se habilitan las interrupciones, este registro indica a la UART los eventos habilitados.*



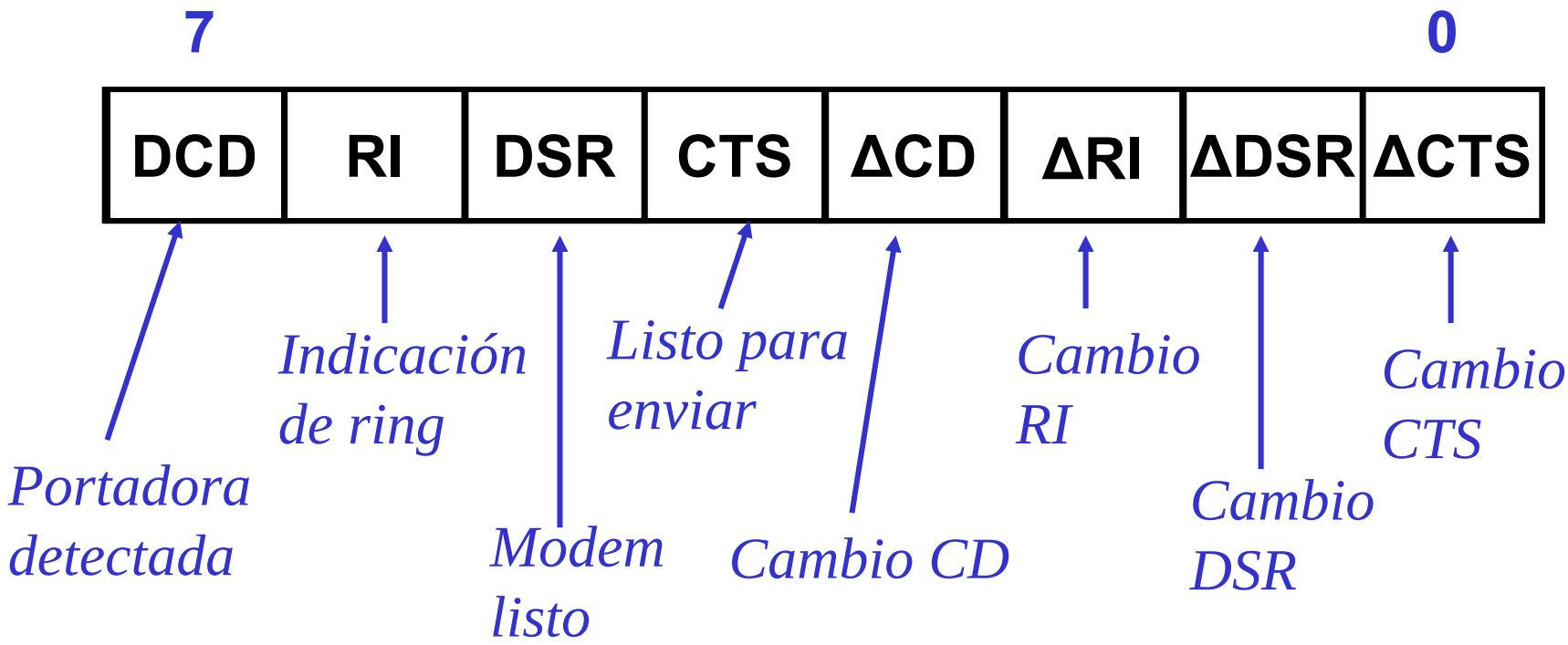
Registro de Estado de Línea

- *Line Status Register: Se usa para obtener información de los datos de transmisión y recepción.*



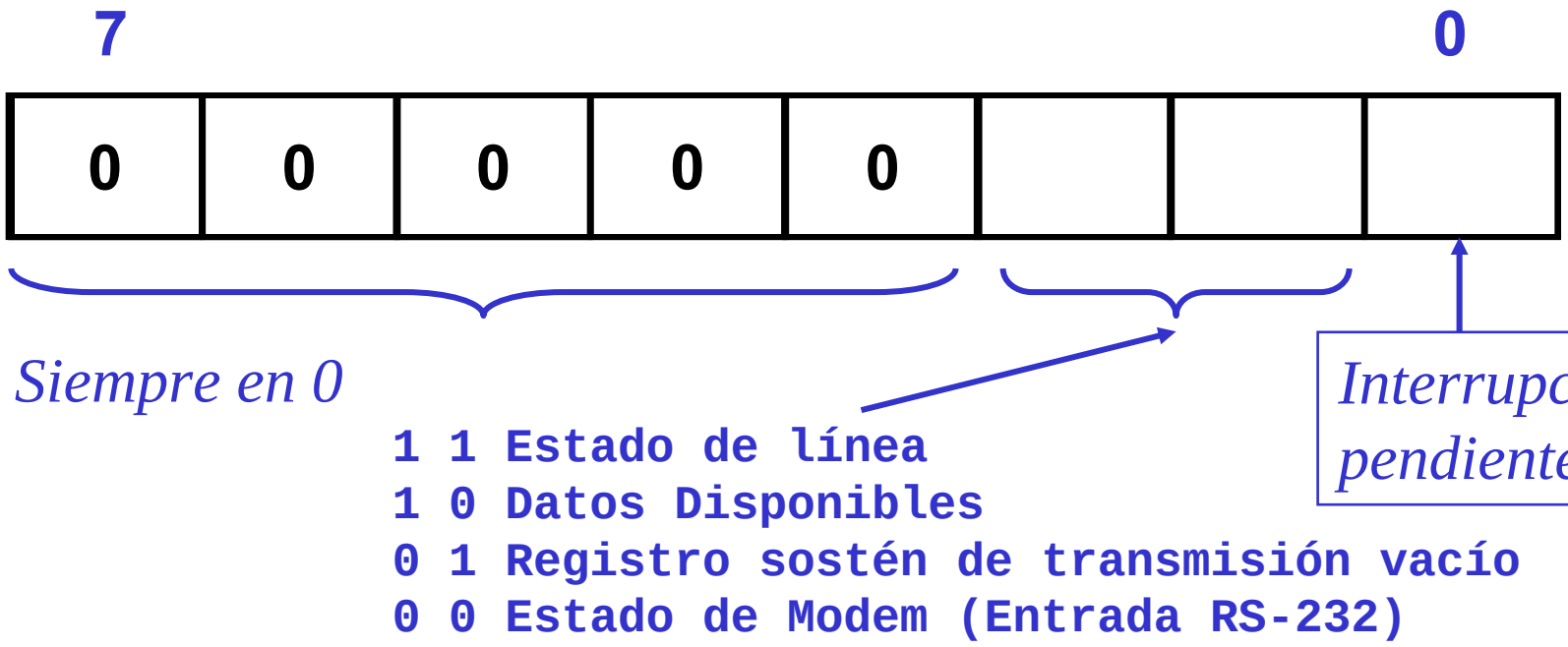
Registro de Estado de Modem

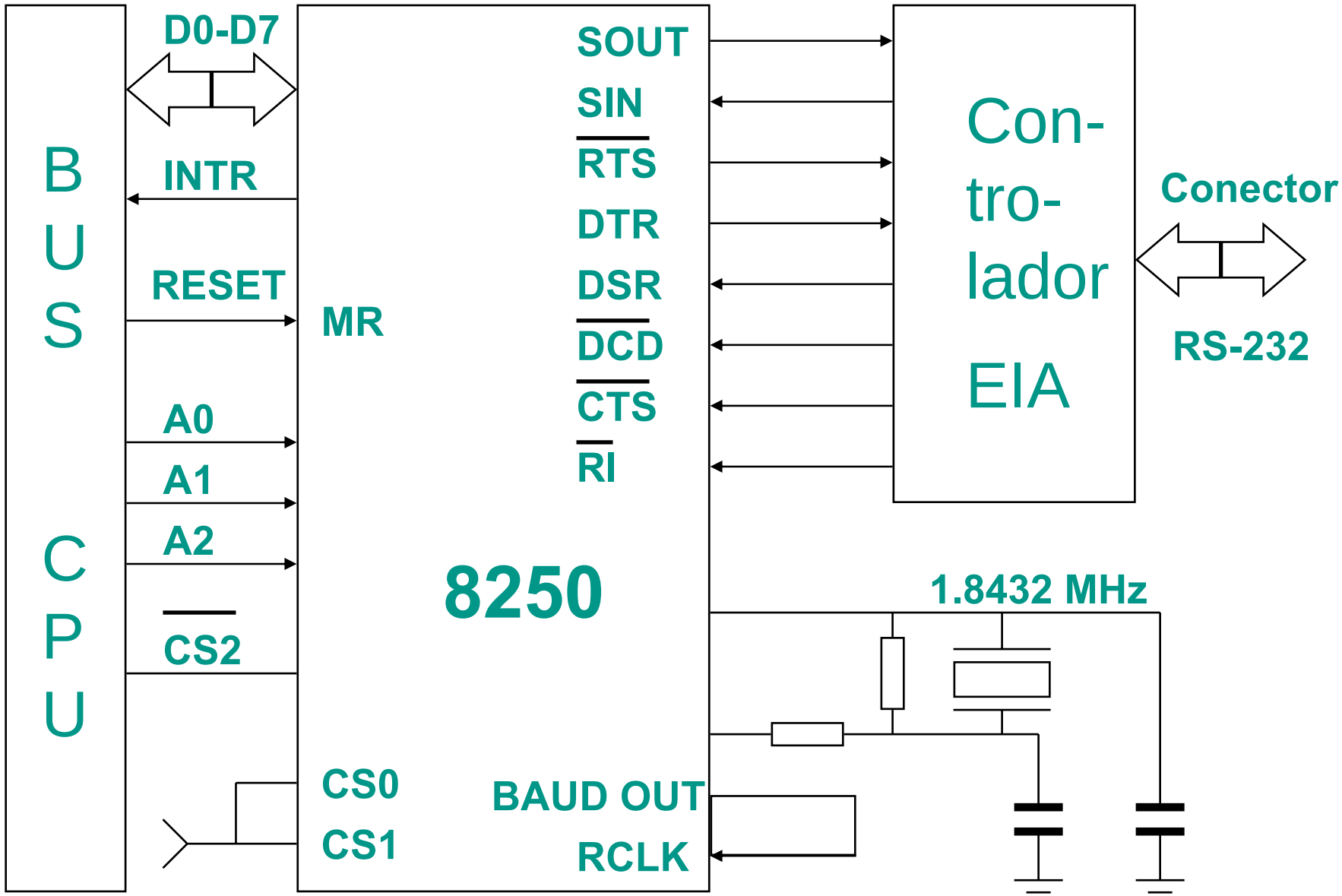
- *Modem Status Register: Mediante este registro conocemos el estados de las líneas externas que conectan al modem.*



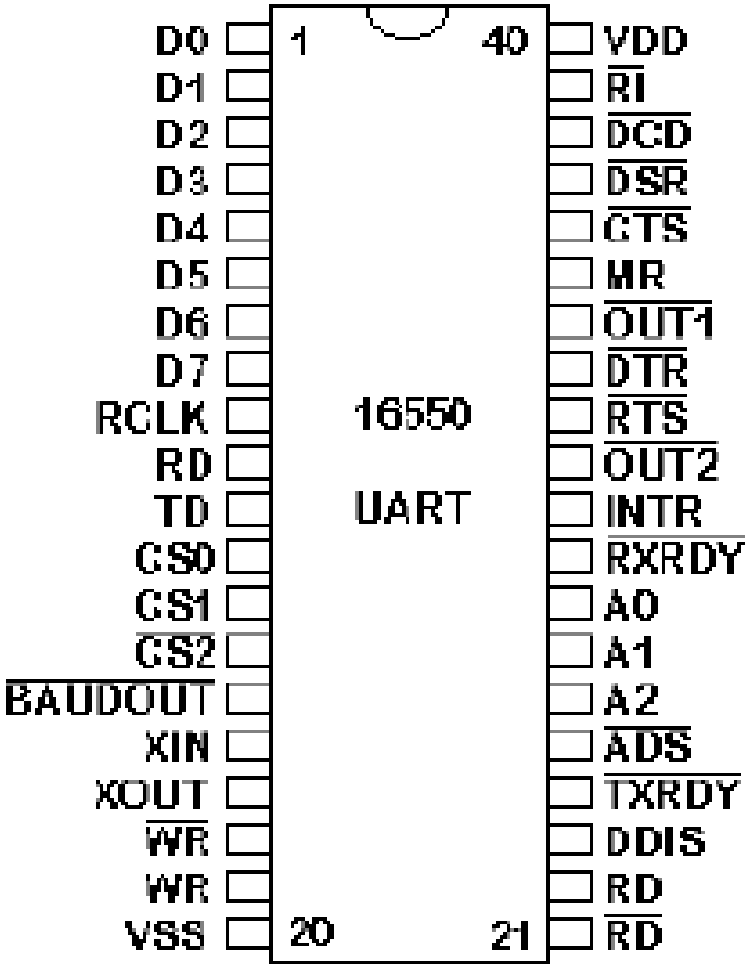
Registro Identificación de Interr.

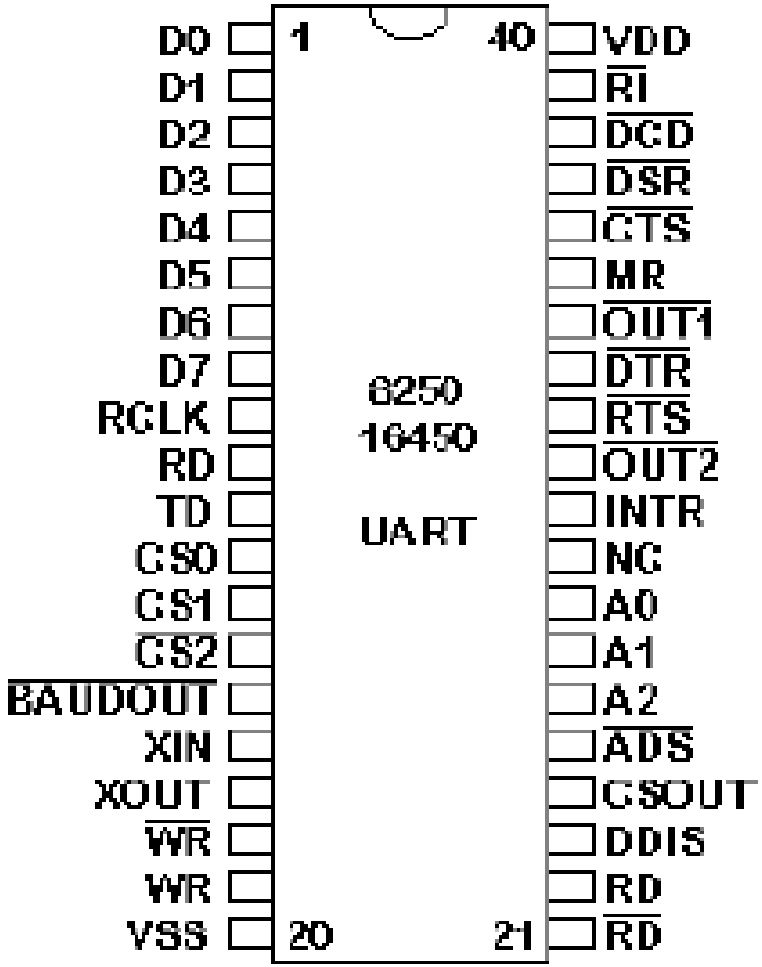
- *Interrupt Identificacion Register: Cuando se produce una interrupción, en este registro se busca la causa.*




BUS CPU
D0-D7
INTR
RESET
A0
A1
A2
CS2
CS0
CS1
MR
8250
SOUT
SIN
RTS
DTR
DSR
DCD
CTS
RI
BAUD OUT
RCLK
Con-
tro-
lador
EIA
Conector
RS-232
1.8432 MHz

Chips UART





Pin Diagrams UARTs 16550, 16450 & 8250

Manejo de la UART en DOS

- *Se utiliza para lectura:*

```
unsigned char inportb (unsigned portid);
```

portid: posición de memoria (ej. 0x3F8).

- *Se utiliza para escritura:*

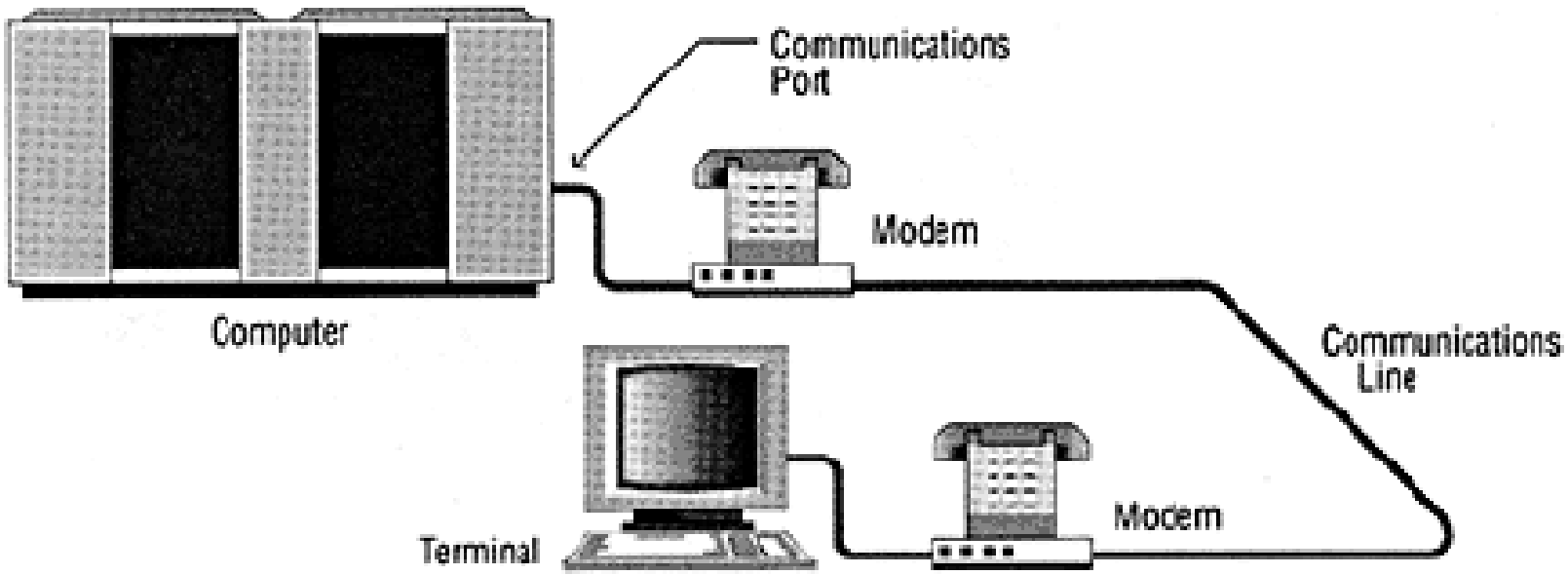
```
void outportb(unsigned portid,
              unsigned char value);
```

value: valor a escribir en esa posición.

Puerto Serie en Linux

- *En todos los Unix, el puerto serie ha sido una pieza fundamental, ya que en un principio toda la comunicación con el exterior se hacía a través de este puerto.*
- *Esto hace que se disponga de una cantidad de posibilidades enorme para su control, pero también implica un proceso de selección y estudio mayor para elegir las funciones adecuadas a cada trabajo.*

Puerto Serie en Linux



Puerto Serie en Linux

- *La filosofía básica de Unix implica considerar a la comunicación como archivos especiales.*
- *Los puertos serie son archivos que están bajo el directorio /dev y se denominan:*

```
/dev/ttyS0      (COM1 de Win/DOS)
/dev/ttyS1      (COM2 de Win/DOS)
/dev/ttyS2      (COM3 de Win/DOS)
/dev/ttyS3      (COM4 de Win/DOS)
```

Manejo de Puerto Serie en Linux

- *Para averiguar el estado de un puerto, usamos el comando (a nivel de SO):*

  `setserial /dev/ttySx (x = 0, 1, 2, 3)`

  Respuesta ejemplo:

  ```
  /dev/ttyS0, UART: 16550A, Port: 0x03f8, IRQ: 4
  ```

- *Averiguación más completa:*

  `setserial -a /dev/ttySx (x = 0, 1, 2, 3)`

  ```
  /dev/ttyS0, Line 0, UART: 16550A, Port: 0x03f8, IRQ: 4
        Baud_base: 115200, close_delay: 50, divisor: 0
        closing_wait: 3000
        Flags: spd_normal skip_test
  ```

Manejo de Puerto Serie en Linux

- ***setserial*** *es un comando que permite indagar o configurar un puerto serie.*
- *Otro comando utilizado para el manejo del puerto desde el Sistema Operativo es* ***stty***

```
stty -F /dev/ttySx (x = 0, 1, 2, 3)
```

Respuesta ejemplo:

```
speed 2400 baud; line = 0;
-brkint -imaxbel
```

Manejo de Puerto Serie en Linux

SETSERIAL(8) SETSERIAL(8)

NAME

setserial - get/set Linux serial port information

SYNOPSIS

setserial [**-abqvVWz**] device [parameter1 [arg]] ...

setserial -g [**-abGv**] device1 ...

DESCRIPTION

setserial is a program designed to set and/or report the configuration information associated with a serial port. This information includes what I/O port and IRQ a particular serial port is using, and whether or not the break key should be interpreted as the Secure Attention Key, and so on.

Manejo de Puerto Serie en Linux

```
STTY(1)                    User Commands                    STTY(1)

NAME
       stty - change and print terminal line settings

SYNOPSIS
       stty [-F DEVICE] [--file=DEVICE] [SETTING]...
       stty [-F DEVICE] [--file=DEVICE] [-a|--all]
       stty [-F DEVICE] [--file=DEVICE] [-g|--save]

DESCRIPTION
       Print or change terminal characteristics.

       -a, --all
              print all current settings in human-readable form

       -g, --save
              print all current settings in a stty-readable form

       -F, --file=DEVICE
              open and use the specified DEVICE instead of stdin
```

Manejo de Puerto Serie en Linux

- *Para la configuración inicial (línea de transmisión) es conveniente utilizar la estructura termios y sus funciones asociadas.*
- *Para el control de modem utilizamos la función ioctl() (Input/Output ConTroL).*
- *Para lectura y escritura utilizamos las funciones read() y write(). Estas trabajan a bajo nivel.*

Estructura termios

- ***termios*** *es una estructura que contiene los parámetros de control del puerto, tiene la siguiente configuración:*

```
struct termios {
    tcflag_t c_iflag; /* parámetros de modo entrada */
    tcflag_t c_oflag; /* parámetros de modo salida */
    tcflag_t c_cflag; /* parámetros de modo control */
    tcflag_t c_lflag; /* parámetros de modo local */
    cc_t c_line;      /* disciplina de la línea */
    cc_t c_cc[NCCS];  /* caracteres de control */
};
```

Estructura termios

- *c_iflag: controla la forma en que la terminal maneja la entrada. Es un OR inclusivo de 11 banderas.*
- *c_oflag: controla la forma en que la terminal maneja la salida. Es un OR inclusivo de 4 banderas.*
- *c_cflag: especifica información de control de hardware para la terminal (ej. paridad). Es un OR inclusivo de 7 banderas*

Estructura termios

- *c_lflag: controla las funciones de edición de la terminal (ej. eco). Es un OR inclusivo de 9 banderas.*
- *c_cc[NCCS]: Mantiene los valores de caracteres que tienen un significado especial para los drivers de la terminal. Es un OR inclusivo de 9 banderas.*

Banderas de c_cflag

- *c_cflag tiene las banderas siguientes, para el control de línea:*

field	flag	description
c_cflag	CSIZE	character size (CS5–CS8 for 5 to 8 bits, respectively)
	CSTOPB	send two stop bits, else one
	CREAD	enable receiver
	PARENB	enable parity
	PARODD	odd parity, else even
	HUPCL	hang up on last close
	CLOCAL	ignore modem status lines

Estructura termios

- ***termios** también es un conjunto de funciones que modifican los parámetros de la estructura configurando el puerto.*

Obtener la estructura termios de un puerto

```
int tcgetattr( int fd, struct termios *termios_p );
```

Escribir la estructura termios a un puerto

```
int tcsetattr( int fd , int optional_actions,
               struct termios *termios_p );
```

Estructura termios

Obtención de la velocidad de un puerto

```
speed_t cfgetospeed( struct termios *termios_p );
```

Fijación de la velocidad en baudios de un puerto

```
int cfsetospeed( struct termios *termios_p,
                 speed_t speed );
```

Envío de break a puerto con duración `duration` *en ms.*

```
int tcsendbreak( int fd, int duration );
```

Acciones opcionales:

TCSANOW *- el cambio ocurre inmediatamente*

TCSADRAIN *- el cambio se produce no bien termina la salida actual*

CTSAFLUSH *- el cambio se produce no bien termina la salida actual toda la entrada recibida pero no leida se descarta antes del cambio.*

Estructura termios

Datos de transmisión

Ejemplo 8N1:

```
struct termios opciones;
opciones.c_cflag &= ~PARENB; // Sin paridad
opciones.c_cflag &= ~CSTOPB; // 1 bit de parada
opciones.c_cflag &= ~CSIZE;  // \
opciones.c_cflag |= CS8;     // / 8 bits de datos
```

Ejemplo 7E1:

```
struct termios opciones;
opciones.c_cflag |= PARENB;  // Con paridad
opciones.c_cflag &= ~PARODD; // Paridad par
opciones.c_cflag &= ~CSTOPB; // 1 bit de parada
opciones.c_cflag &= ~CSIZE;  // \
opciones.c_cflag |= CS7;     // / 7 bits de datos
```

Activación y desactivación de flags

- *El objetivo es activar o desactivar una señal en particular sin afectar el resto.*
- *Poner en cero un bit:*

```
opciones.c_cflag &= ~CSTOPB;
```

- *Poner en uno un bit:*

```
opciones.c_cflag |= CSTOPB;
```

- *Conmutar un bit:*

```
resultado ^= TIOCM_RTS;
```

Bibliotecas utilizadas

```
#include <termios.h>        /* Estructura de control termios */

#include <sys/ioctl.h>      /* Constantes para la función ioctl */

#include <fcntl.h>          /* Definiciones de control de archivos */

#include <errno.h>          /* Definiciones de control de errores */
```

```
/* Programa que inicializa un puerto serie abriéndolo como archivo.
   Imprime la velocidad y características de transmisión actuales.
   Configura la velocidad y características de transmisión e imprime
   el estado en que queda el puerto. */

#include <fcntl.h>      /* Definiciones de control de archivos */
#include <errno.h>      /* Definiciones de control de errores */
#include <sys/ioctl.h>  /* Constantes para ioctl */
#include <termios.h>    /* Estructura de control termios */

void estadolinea( char *puerto, struct termios opciones );

int main( void ) {
  int fd;
  char puerto[] = "/dev/ttyS0"; // COM1
  struct termios opciones;

  fd = open(puerto, O_RDWR | O_NOCTTY | O_NDELAY | O_NONBLOCK);
  if (fd == -1) {
    perror("open_port");
    exit( 1 );
  }
  else
    fcntl(fd, F_SETFL, 0); // funciona en forma síncrona
                           // con FNDELAY lo hace en forma asíncrona
```

U01.c

Función fcntl()

```
FCNTL(2)              Linux Programmer's Manual              FCNTL(2)

NAME
       fcntl - manipulate file descriptor

SYNOPSIS
       #include <unistd.h>
       #include <fcntl.h>

       int fcntl(int fd, int cmd);
       int fcntl(int fd, int cmd, long arg);
       int fcntl(int fd, int cmd, struct flock *lock);

DESCRIPTION
       fcntl  performs one of various miscellaneous operations on
       fd.  The operation in question is determined by cmd.

   Handling close-on-exec
       F_DUPFD
              Find the lowest numbered available file  descriptor
```

```
// Estado de linea por la estructura termios
tcgetattr( fd, &opciones );
printf( "Antes de configurar: \n" );
estadolinea( puerto, opciones );

// Fijando velocidad del puerto
cfsetispeed( &opciones, B2400 );

// Habilitación del receptor y modo local
opciones.c_cflag |= (CLOCAL | CREAD);

// Tamaño de palabra
opciones.c_cflag &= ~CSIZE; /* Enmascara bits de tamaño de palabra */
opciones.c_cflag |= CS8;    /* Selecciona 8 bits */

// Esto es para fijar 1 stop bit
opciones.c_cflag &= ~CSTOPB;

// Esto es para fijar 2 stop bits.
//opciones.c_cflag |= CSTOPB;

// Esto es para fijar NO paridad (N)
opciones.c_cflag &= ~PARENB;

// Esto es para fijar paridad par (E)
//opciones.c_cflag |= PARENB;
//opciones.c_cflag &= ~PARODD;
```

```
  // Esto es para fijar paridad impar (O)
  //opciones.c_cflag |= PARENB;
  //opciones.c_cflag |= PARODD;

  // Escritura de la configuración en el puerto
  tcsetattr( fd, TCSANOW, &opciones );

  // Lectura de la configuración del puerto
  tcgetattr( fd, &opciones );

  printf( "Después de configurar: \n" );
  estadolinea( puerto, opciones );

  close( fd );
}
```

Ej. de Salida

```
Antes de configurar:
/dev/ttyS0: 8N1 9600 bps
Después de configurar:
/dev/ttyS0: 8N1 2400 bps
```

```
void estadolinea( char *puerto, struct termios opciones ) {
  int speed;

  printf( "%s: ", puerto );
  // Bits de cada palabra
  if( (opciones.c_cflag & CSIZE) == CS5) printf( "5" );
  if( (opciones.c_cflag & CSIZE) == CS6) printf( "6" );
  if( (opciones.c_cflag & CSIZE) == CS7) printf( "7" );
  if( (opciones.c_cflag & CSIZE) == CS8) printf( "8" );

  // Paridad
  if( (opciones.c_cflag & PARENB) == 0) printf( "N" );
  else {
    if( (opciones.c_cflag & PARODD) == PARODD) printf( "O" );
    else printf( "E" );  // paridad par
  }

  // Bits de parada
  if( (opciones.c_cflag & CSTOPB) == CSTOPB) printf( "2" );
  else printf( "1" );
```

```
  // Velocidad de transmisión
  printf( " " );
  speed = cfgetispeed( &opciones );
  switch( speed ) {
    case B50: printf( "50" ); break;
    case B75: printf( "75" ); break;
    case B110: printf( "110" ); break;
    case B134: printf( "134" ); break;
    case B150: printf( "150" ); break;
    case B200: printf( "200" ); break;
    case B300: printf( "300" ); break;
    case B600: printf( "600" ); break;
    case B1200: printf( "1200" ); break;
    case B1800: printf( "1800" ); break;
    case B2400: printf( "2400" ); break;
    case B4800: printf( "4800" ); break;
    case B9600: printf( "9600" ); break;
    case B19200: printf( "19200" ); break;
    case B38400: printf( "38400" ); break;
    case B57600: printf( "57600" ); break;
    case B115200: printf( "115200" ); break;
    default: printf ( "Error" ); break;
  }
  printf( " bps\n" );
}
```

Función ioctl()

- *ioctl() permite controlar los parámetros del dispositivo representado por un archivo especial (en nuestro caso el puerto serie).*

```
#include <sys/ioctl.h>
int ioctl(int d, int request, ...);
```

Ejemplo:

```
ioctl(fd, TIOCSERGETLSR, (int) &resultado);
```

fd: entero descriptor de archivo

TIOCSERGETLSR: requerimiento al dispositivo

resultado: puntero void a área de memoria

Función ioctl()

IOCTL(2) Linux Programmer's Manual IOCTL(2)

NAME

ioctl - control device

SYNOPSIS

```
#include <sys/ioctl.h>

int ioctl(int d, int request, ...);
```

DESCRIPTION

The **ioctl** function manipulates the underlying device parameters of special files. In particular, many operating characteristics of character special files (e.g. terminals) may be controlled with **ioctl** requests. The argument *d* must be an open file descriptor.

The second argument is a device-dependent request code. The third argument is an untyped pointer to memory. It's traditionally **char** **argp* (from the days before **void** * was valid C), and will be so named for this discussion.

Control de Modem en Linux

- *Usamos ioctl() con los comandos siguientes:*
  - *TIOCMGET: se usa para conocer el estado de las líneas de control de modem (DTR y RTS)*
  - *TIOCMBIS: se usa para activar el valor indicado en el tercer argumento (DTR ó RTS)*
  - *TIOCMBIC: se usa para desactivar el valor indicado en el tercer argumento. (DTR....)*

Control de Modem en Linux

- *Usamos ioctl() con los comandos siguientes (cont.):*
  - *TIOCMSET: se usa para establecer el estado de las líneas de control de modem (DTR y RTS)*
  - *TIOCSBRK: coloca un break en TxD (lleva la línea a positivo)*
  - *TIOCCBRK: cancela un break en TxD (lleva la línea a negativo)*

Control de Modem en Linux

- *Constantes utilizadas (3er. argumento)*

```
TIOCM_DTR    DTR    0x002
TIOCM_RTS    RTS    0x004
TIOCM_CTS    CTS    0x020
TIOCM_CAR    DCD    0x040
TIOCM_CD     DCD    0x040
TIOCM_RNG    RI     0x080
TIOCM_RI     RI     0x080
TIOCM_DSR    DSR    0x100
```

TIOCM_DTR	DTR	0x002
TIOCM_RTS	RTS	0x004
TIOCM_CTS	CTS	0x020
TIOCM_CAR	DCD	0x040
TIOCM_CD	DCD	0x040
TIOCM_RNG	RI	0x080
TIOCM_RI	RI	0x080
TIOCM_DSR	DSR	0x100

```
/* Programa que inicializa un puerto serie abriéndolo como archivo,
   obtiene el estado de modem y lo imprime,  a continuación baja y
   sube las líneas DTR y RTS mostrando los códigos resultantes a
   partir de la lectura de estado de modem. */

#include <fcntl.h>     /* Definiciones de control de archivos */
#include <errno.h>     /* Definiciones de control de errores */
#include <sys/ioctl.h> /* Constantes para función ioctl */

void imprimestado( int estado, char *puerto );
```

U03.c

```
int main( void ) {
  int fd, estado, resultado;
  char puerto[] = "/dev/ttyS0"; // COM1
  long tiempo = 500000, i;

  fd = open(puerto, O_RDWR | O_NOCTTY | O_NDELAY);
  if (fd == -1) {
    perror("open_port");
    exit( 1 );
  }

  estado = ioctl( fd, TIOCMGET, (int) &resultado );
  if( estado == -1 ) {
    perror("ioctl");
    exit( 1 );
  }
```

```
imprimestado( resultado, puerto );
usleep( tiempo );

for( i=0; i<10; i++ ) {
  printf( "Cambiando RTS\n" );
  resultado ^= TIOCM_RTS;
  estado = ioctl( fd, TIOCMSET, (int) &resultado );
  if( estado == -1 ) {
    perror("ioctl");
    exit( 1 );
  }

  estado = ioctl( fd, TIOCMGET, (int) &resultado );
  if( estado == -1 ) {
    perror("ioctl");
    exit( 1 );
  }
  imprimestado( resultado, puerto );
  usleep( tiempo );

  printf( "Cambiando DTR\n" );
  resultado ^= TIOCM_DTR;
  estado = ioctl( fd, TIOCMSET, (int) &resultado );
  if( estado == -1 ) {
    perror("ioctl");
    exit( 1 );
  }
```

```
    estado = ioctl( fd, TIOCMGET, (int) &resultado );
    if( estado == -1 ) {
      perror("ioctl");
      exit( 1 );
    }
    imprimestado( resultado, puerto );
    usleep( tiempo );
  }
  close( fd );
}

void imprimestado( int estado, char *puerto ) {
  printf( "Estado puerto %s:  ", puerto );
  if( estado & TIOCM_DTR ) printf( "DTR " ); else printf( "    " );
  if( estado & TIOCM_RTS ) printf( "RTS " ); else printf( "    " );
  if( estado & TIOCM_DSR ) printf( "DSR " ); else printf( "    " );
  if( estado & TIOCM_CTS ) printf( "CTS " ); else printf( "    " );
  if( estado & TIOCM_CD  ) printf( "CD "  ); else printf( "   " );
  if( estado & TIOCM_RI  ) printf( "RI "  ); else printf( "   " );
  printf( "\n" );
}
```

Lectura y escritura en Linux

- *Usamos para lectura:*

```
ssize_t read(int fd, void *buf,
                size_t count);
```

fd: file descriptor (abierto con open()).
buf: puntero a memoria para almacenar.
count: cantidad de elementos a leer.

***read** devuelve el número de bytes leídos.*

Biblioteca: `<unistd.h>`

Lectura y escritura en Linux

- *Usamos para escritura:*

```
ssize_t write(int fd, const void *buf,
                    size_t count);
```

fd: file descriptor (abierto con open()).

buf: puntero a memoria desde donde se lee.

count: cantidad de elementos a escribir.

***write** devuelve el número de bytes escritos.* *Biblioteca:* `<unistd.h>`

Lectura y escritura en Linux

- *Programa que lee y escribe simultánea-mente en un puerto serie, modo no canó-nico.*
- *Se especifica en la línea de comandos:*

  nombre puerto disciplina velocidad

  ej.: U06 0 8N1 9600

- *Observar la función kbhit(), que emula su homónima de Borland y Turbo C, esta función no existe en forma nativa en Unix*

```
/* Programa que inicializa un puerto serie abriéndolo como archivo.
   Imprime la velocidad y características de transmisión actuales.
   Configura la velocidad y características de transmisión de acuerdo
   a lo ingresado en la línea de comandos e imprime el estado en que
   queda el puerto.
   A continuación envía al puerto lo que recibe por teclado y escribe
   en pantalla lo que recibe desde el puerto */

#include <fcntl.h>      /* Definiciones de control de archivos */
#include <errno.h>      /* Definiciones de control de errores */
#include <sys/ioctl.h>  /* Constantes para ioctl */
#include <termios.h>    /* Estructura de control termios */
#include <unistd.h>
```

U06.c

```
static struct termios initial_settings, new_settings;
static int peek_character = -1;

void estadolinea( char *puerto, struct termios opciones );
int velocidad( int speed );
void init_keyboard();
void close_keyboard();
int readch(); int kbhit();

int main( int argc, char *argv[] ) {
  int fd;                       /* descriptor archivo del puerto */
  char puerto[12] = "/dev/ttyS";
  int longpal;                  /* longitud de palabra */
  int estado, resultado;
  char letrain=0, letraout=0;   /* caracteres de lectura escritura */
  struct termios opciones;
```

```
int ch = 0;                     /* para kbhit() */
int cuenta;                     /* caracteres leidos en el puerto */

/* CONTROL DE CONSISTENCIA DE LA LINEA DE COMANDOS */
if( argc < 4 ) {
  printf( "\nUso: %s puerto[0..3] disciplina[ej. 8N1] velocidad[bps]\n\n",
          argv[0] );
  exit(1);
}
strcat( puerto, argv[1] );
printf( "Puerto: %s\n", puerto );

if( (atoi(argv[1]) < 0) || (atoi(argv[1]) > 3) ) {
  printf( "\nEl puerto debe ser entre 0 y 3\n\n" );
  exit(1);
}

if( (atoi(&argv[2][0]) < 5) || (atoi(&argv[2][0]) > 8) ) {
  printf( "Chars por palabra debe estar entre 5 y 8\n" );
  exit( 1 );
}

if( (toupper(argv[2][1]) != 'N') && (toupper(argv[2][1]) != 'E') &&
    (toupper(argv[2][1]) != 'O') ) {
  printf( "Paridad debe ser N, E, O\n" );
  exit( 1 );
}

if( (atoi(&argv[2][2]) < 1) || (atoi(&argv[2][2]) > 2) ) {
  printf( "Bits de stop debe ser 1 ó 2\n" );
  exit( 1 );
```

```
  }
  /* FIN CONTROL DE CONSISTENCIA DE LA LINEA DE COMANDOS */

  // Apertura del puerto
  fd = open(puerto, O_RDWR | O_NOCTTY | O_NDELAY | O_NONBLOCK);
  if (fd == -1) {
    perror("open_port");
    exit( 1 );
  }
  else
//    fcntl(fd, F_SETFL, 0); // funciona en forma síncrona
    fcntl(fd, F_SETFL, FNDELAY); // funciona en forma asíncrona

  /* SUBIENDO DTR Y RTS */
  estado = ioctl( fd, TIOCMGET, (int) &resultado );
  if( estado == -1 ) {
    perror("ioctl");
    exit( 1 );
  }

  // Lectura de estado de linea por la estructura termios
  tcgetattr( fd, &opciones );
  printf( "Antes de configurar: \n" );
  estadolinea( puerto, opciones );

  // Fijando velocidad del puerto
  cfsetispeed( &opciones, velocidad( atoi(argv[3]) ) );

  // Habilitación del receptor y modo local
  opciones.c_cflag |= (CLOCAL | CREAD);
```

```
// Tamaño de palabra
opciones.c_cflag &= ~CSIZE; /* Mask the character size bits */
switch( atoi(&argv[2][0]) ) {
  case 5: longpal = CS5; break;
  case 6: longpal = CS6; break;
  case 7: longpal = CS7; break;
  case 8: longpal = CS8; break;
}
opciones.c_cflag |= longpal;

// Fijando la cantidad de stop bits
if( atoi(&argv[2][2]) == 1 )
  opciones.c_cflag &= ~CSTOPB;
else
  opciones.c_cflag |= CSTOPB;

// Fijando la paridad
if( toupper(argv[2][1]) == 'N' )
  opciones.c_cflag &= ~PARENB;

if( toupper(argv[2][1]) == 'E' ) {
  opciones.c_cflag |= PARENB;
  opciones.c_cflag &= ~PARODD;
}

if( toupper(argv[2][1]) == 'O' ) {
  opciones.c_cflag |= PARENB;
  opciones.c_cflag |= PARODD;
}
// Condiciones para que lea de a un elemento en modo NO CANONICO
opciones.c_lflag &= ~(ECHO | ICANON | ISIG);
```

```
opciones.c_oflag &= ~OPOST;
opciones.c_cc[VTIME] = 0;
opciones.c_cc[VMIN] = 1;

// Fijación de los parámetros el puerto
tcsetattr( fd, TCSANOW, &opciones );

// Obtención de los parámetros el puerto
tcgetattr( fd, &opciones );

printf( "Después de configurar: \n" );
estadolinea( puerto, opciones );

// Lazo de lectura y escritura en el puerto
init_keyboard();
while( (letrain != 4) && (letraout != 4) ) {  // <CTRL-D>
      if(kbhit()) {
              letraout = readch();
              write( fd, &letraout, 1 );
              if(letraout == 10) {
                      letraout = 13;
                      write( fd, &letraout, 1 );
              }
              fflush( NULL );
      }
      cuenta=read(fd, &letrain, 1);
      if( cuenta > 0 ) {
              write( 0, &letrain, 1 );
              fflush( NULL );
      }
}
```

```
  close_keyboard();
  close( fd );
  exit(0);
}

void estadolinea( char *puerto, struct termios opciones ) {
  int speed;

  printf( "%s: ", puerto );
  // Bits de cada palabra
  if( (opciones.c_cflag & CSIZE) == CS5) printf( "5" );
  if( (opciones.c_cflag & CSIZE) == CS6) printf( "6" );
  if( (opciones.c_cflag & CSIZE) == CS7) printf( "7" );
  if( (opciones.c_cflag & CSIZE) == CS8) printf( "8" );

  // Paridad
  if( (opciones.c_cflag & PARENB) == 0) printf( "N" );
  else {
    if( (opciones.c_cflag & PARODD) == PARODD) printf( "O" );
    else printf( "E" );  // paridad par
  }

  // Bits de parada
  if( (opciones.c_cflag & CSTOPB) == CSTOPB) printf( "2" );
  else printf( "1" );

  // Velocidad de transmisión
  printf( " " );
  speed = cfgetispeed( &opciones );
  switch( speed ) {
    case B50: printf( "50" ); break;
```

```
    case B75: printf( "75" ); break;
    case B110: printf( "110" ); break;
    case B134: printf( "134" ); break;
    case B150: printf( "150" ); break;
    case B200: printf( "200" ); break;
    case B300: printf( "300" ); break;
    case B600: printf( "600" ); break;
    case B1200: printf( "1200" ); break;
    case B1800: printf( "1800" ); break;
    case B2400: printf( "2400" ); break;
    case B4800: printf( "4800" ); break;
    case B9600: printf( "9600" ); break;
    case B19200: printf( "19200" ); break;
    case B38400: printf( "38400" ); break;
    case B57600: printf( "57600" ); break;
    case B115200: printf( "115200" ); break;
    default: printf ( "Error" ); break;
  }
  printf( " bps\n" );
}

int velocidad( int speed ) {
  switch( speed ) {
    case 50: return( B50 );
    case 75: return( B75 );
    case 110: return( B110 );
    case 134: return( B134 );
    case 150: return( B150 );
    case 200: return( B200 );
    case 300: return( B300 );
    case 600:return( B600 );
```

```
    case 1200: return( B1200 );
    case 1800: return( B1800 );
    case 2400: return( B2400 );
    case 4800: return( B4800 );
    case 9600: return( B9600 );
    case 19200: return( B19200 );
    case 38400: return( B38400 );
    case 57600: return( B57600 );
    case 115200: return( B115200 );
    default: printf ( "Error velocidad no permitida\n" );
             exit(1);
             break;
  }
}

void init_keyboard() {
        tcgetattr(0,&initial_settings);
        new_settings = initial_settings;
        new_settings.c_lflag &= ~ICANON;
        new_settings.c_lflag &= ~ECHO;
        new_settings.c_lflag &= ~ISIG;
        new_settings.c_cc[VMIN] = 1;
        new_settings.c_cc[VTIME] = 0;
        tcsetattr(0, TCSANOW, &new_settings);
}

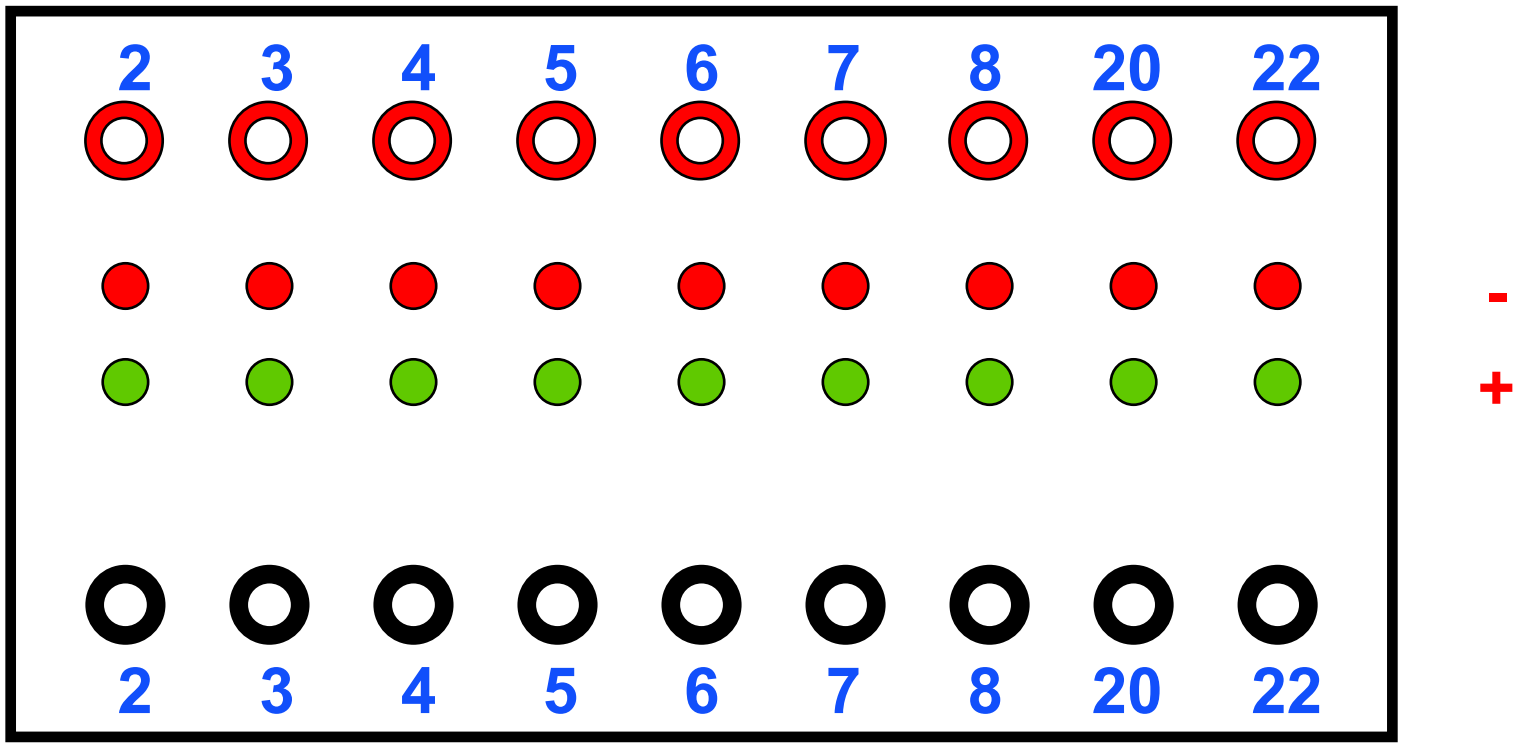
void close_keyboard() {
        tcsetattr(0, TCSANOW, &initial_settings);
}
```

```
int kbhit() {
        char ch;
        int nread;
        if(peek_character != -1)
                return 1;
        new_settings.c_cc[VMIN]=0;
        tcsetattr(0, TCSANOW, &new_settings);
        nread = read(0,&ch,1);
        new_settings.c_cc[VMIN]=1;
        tcsetattr(0, TCSANOW, &new_settings);
        if(nread == 1) {
                peek_character = ch;
                return 1;                int kbhit();
        }
        return 0;
}

int readch() {
        char ch;
        if(peek_character != -1) {
                ch = peek_character;
                peek_character = -1;
                return ch;
        }
        read(0,&ch,1);
        return ch;
}
```

Caja de Conexiones

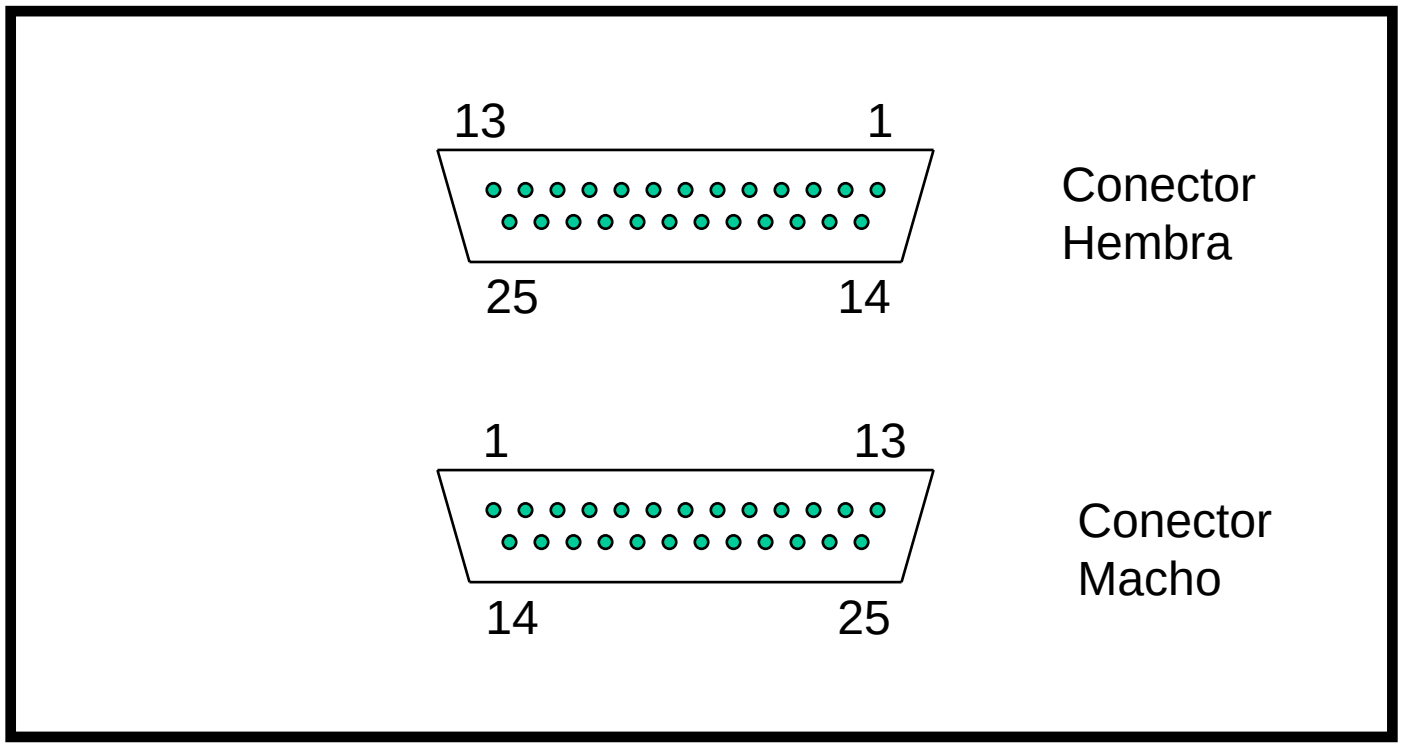
Parte Anterior



Break Out Box

Caja de Conexiones

Parte Posterior



Break Out Box

Norma RS-449

- *Básicamente es similar al RS-232, pero permite más distancia y tiene menos modulación cruzada.*
- *Describe las especificaciones mecánicas y los circuitos, pero no las señales eléctricas.*
- *Usa un conector DB-37, ya que permite circuitos balanceados y no balanceados.*

Norma RS-449

- *Las especificaciones eléctricas están cubiertas por las normas RS-422-A y RS-423-A, para circuitos balanceados y no balanceados.*